Università degli Studi

BlueAgent

Relatore: Pierpaolo Loreti

Correlatore: Ing. Francesco Mancini

Candidato:
Davide Luci

Anno Accademico 2023-2024

Indice

1 Introduzione **1**

1.1 Introduzione al Problema 1

1.2 Oracoli e Meccanismi di Verifica 1

1.2.1 Verifica Statica 2

1.2.2 Verifica Dinamica tramite sistema esterno 3

1.2.3 Verifica Dinamica tramite injection dei comandi di verifica . . 3

1.2.4 Clonazione del sistema in una rete separata 4

1.3 Cyber-Range: lo stato dell'arte 4

1.3.1 CR Alternativi: kCTF 5

1.3.2 CR Alternativi: CTFBox 8

1.3.3 CR Alternativi: CyTrONE 9

1.3.3.1 Valutazione delle Performance di CyTrONE 13

1.3.4 CR Alternativi: CyberRangeCZ 16

1.3.5 CR Alternativi: CyberPatrion 19

2 BlueAgent **20**

2.1 2. Introduzione a BlueAgent 20

2.2 Attori 21

2.2.1 CTF Manager (attore) 21

2.3 CTF Manager (sistema) 21

2.3.1 Giocatore (attore esterno per CTF Manager) 21

2.3.2 Autore della sfida (attore esterno per CTF Manager) 21

2.3.3 Amministratore della competizione (attore esterno per CTF Manager) 22

2.4 Use Cases per CTF Manager 22

2.4.1 `UC-01`: Deploy dell'infrastruttura di rete per il team k 22

2.4.2 `UC-02`: Deploy della challenge per il team k 23

2.4.3 `UC-03`: Verifica della challenge per il team k 23

2.4.4 `UC-04`: Deploy di un unico container per il team k [TODODOD] 24

2.4.5 `UC-05`: Dismissione e rilascio di un unico container per il team k [TODO!!!@#@!#] 25

2.4.6 `UC-06`: Dismissione e rilascio di una challenge per il team k . . 26

2.4.7 UC-07: Dismissione e rilascio di un intero team k 26
2.4.8 UC-08: Compilazione e Import di un'immagine per container . 27
2.4.9 UC-09: Dismissione di un'immagine per container [TPDOAODOD-SA] 28
2.4.10 UC-10: Ispezione dei Peer WireGuard 28
2.4.11 UC-11: Ispezione delle macchine in esecuzione 29
2.4.12 UC-12: Healthcheck 29
2.4.13 UC-13: Registrazione e configurazione WireGuard per un membro del team k 30
2.4.14 UC-14: Revoca di un membro WireGuard per il team k 30
2.5 Requisiti di Sistema 31
2.5.1 Piattaforma Host e Infrastruttura (SR-HST) 31
2.5.2 Motore di Containerizzazione (SR-ENG) 31
2.5.3 Rete Ibrida e Connettivita (SR-NET) 32
2.5.4 Motore di Verifica OVN Sandbox (SR-VRF) 32
2.5.5 Control Plane, Concorrenza e Persistenza (SR-CON) 32
2.5.6 Pipeline Immagini Distrobuilder (SR-IMG) 33
2.6 Esposizione dell'Architettura 33
2.6.1 rete production-k 33
2.6.2 rete verify-k 33
2.6.3 vista integrata (merge produzione e verifica) 35
2.7 Difesa dell'architettura 35
2.7.1 4. Destinatari e Casi d'Uso 38
2.7.2 5. Critica e Limiti Architetturali 39
2.8 Benchmarks 39

3 Architetture Alternative **40**
3.0.1 PID namespace separato per la verifica 40
3.0.2 network namespace separato per la verifica 41
3.0.2.1 Conclusione 42
3.0.3 Esecuzione in-memory con mascheramento 42

4 Appendice **44**
4.1 linux network: net_device 44
4.2 linux network: net_device e sk_buff 44
4.2.1 network device drivers 44
4.2.2 Socket Buffer 45
4.2.3 Netfilter 45
4.2.4 Connection Tracking 47
4.3 TODO: NAT e masquarade 47
4.4 bridge 47

4.5 linux bridge . . . 48
4.5.1 STP . . . 49
4.6 cosa approfondire . . . 49
4.6.1 shadow bridge . . . 49
4.6.2 dual nic . . . 49
4.6.3 PID namespace separato . . . 50
4.6.4 network namespace separato . . . 50
4.6.5 esecuzione in memoryu . . . 50
4.7 VPN wireguard . . . 51
4.8 SDN . . . 52
4.9 ZFS . . . 53
4.9.1 pool . . . 54
4.9.2 transaction groups . . . 54
4.9.3 vantaggi di pool e transazioni . . . 55
4.9.4 quando ZFS costa in memoria? . . . 55
4.10 struttura di una pool . . . 55
4.10.1 `vdev` . . . 55
4.10.2 quindi a che serve un vdev? . . . 56
4.11 dataset . . . 56
4.12 snapshots . . . 57
4.12.1 clones . . . 57
4.12.1.1 nsjail . . . 57
4.12.1.2 namespaces . . . 58
4.12.1.3 seccomp bpf . . . 58
4.12.1.4 cgroups . . . 58

Elenco delle figure **59**

Elenco dei codici **59**

Elenco degli algoritmi **60**

Capitolo 1

Introduzione

1.1 Introduzione al Problema

Nel lavoro di tesi, vogliamo gestire una competizione CTF.

Di solito, in una competizione CTF, uno o piu' utenti (eventualmente organizzati in team) si sfidano per risolvere un'insieme di challenge. Generalmente una challenge vuol dire: recuperare una **flag**.

Ci sono vari tipi di competizioni CTF, tra cui:

- **Jeopardy**: gli utenti devono risolvere vari problemi, la cui flag e' di solito una stringa del tipo `{hai_trovato_la_flag}`
- **Attacco e Difesa**: e' un tipo di competizione dove ogni team, deve recuperare una o piu flag attaccando i sistemi dei team avversare, mentre ognuno si occupa di difendere il proprio sistema.

In particolare, con il nostro lavoro, vogliamo gestire uno o piu' sistemi vulnerabili ad un qualche tipo di attacco.

La flag si ottiene difendendo il sistema, ossia *"patchando"* la vulnerabilita. Dunque la flag non e' piu una stringa come nelle **Jeopardy**, ma piuttosto e': uno stato che deve raggiungere il sistema target mediante il susseguirsi di azioni che il partecipante applica quest'ultimo.

Nel lavoro si tiene conte del fatto che il sistema vulnerabile, spesso, potrebbe essere costituito da uno o piu host/nodi che comunicano tra di loro. Per esempio si condideri il seguente scenario: un server vulnerabile ha bisogno di comunicare con database in esecuzione su un nodo differente da quello del server.

1.2 Oracoli e Meccanismi di Verifica

Per decidere se un team ha recuperato la **flag**, bisogna definire un'***oracolo***, che determini lo stato in cui si trova il sistema target.

L'oracolo e' in grado di accedere al sistema e di fare due azioni:

- puo' leggere la memoria fisica di ogni nodo
- puo' eseguire comandi su qualsiasi nodo

Per ogni tipo di azione eseguita, l'oracolo determina lo stato del sistema confrontando il risultato dell'azione con il valore che l'oracolo si aspettava (ossia i valori vengono validati).

Dunque all'iterno del processo di verifica, il sistema si puo' definire *patchato* se e soltanto se tutte le azioni sono validate rispetto al valore che ci si aspettava in output.

Di seguito sono discussi alcuni meccanismi di verifica che abbiamo identificato.

1.2.1 Verifica Statica

La verifica statica prevede che l'oracolo possa accedere alla memoria fisica, comparando le patch applicate nei file in memoria, ad una patch considerata di riferimento per l'oracolo.

Questo tipo di verifica puo' essere implementato in vari modi:

- **diff**: ad ogni file interessato corrisponde un file di riferimento per l'oracolo. Ogni file viene dunque confrontato, riga per riga, con il suo riferimento per determinare se il team ha applicato la patch correttamente.
- **SAST**: Static Analysis Security Testing. Prevede l'accesso al codice sorgente non compilato in modo da analizzare pattern di codice vulnerabili e fallace logiche.[1]

Tuttavia, SAST non e' un metodo di analisi efficace, sebbene efficiente:

- possono essere identificati nel processo di verifica, vari falsi positivi (e falsi negativi)[1]
- non posso identificare staticamente tutti i tipi di errori di sicurezza [1]
- non posso identificare vulnerabilita all'interno dei binari compilati[1]

Dunque l'analisi statica puo' in alcuni casi risultare inesetta, e non coprire tutti i casi d'uso per quanto riguarda la gestione di competizioni CTF.

[1] `https://aetic.theiaer.org/archive/v4/v4n3/p1.html` 2. Currrent Status of SAST based on Checkmarx

1.2.2 Verifica Dinamica tramite sistema esterno

Questo meccanismo di verifica prevede l'uso di un **orchestratore esterno** alle macchine vulnerabili, ossia l'attaccante, che possa interagire con la vulnerabilita utilizzando la rete locale. L'attaccante orchestra l'esecuzione di uno o piu comandi, con l'obiettivo di inviare il payload all'interno di un pacchetto di rete.

Durante una competizione di questo tipo, tuttavia, i membri dei team sono legittimati ad accedere alla macchina vulnerabile come utente `root`: cosi da permettere l'applicazione delle patch sui file di sistema. Questo implica che i partecipanti dei team siano in grado di eseguire tool come `tcpdump` e `wireshark` per intercettare il traffico di rete e di conseguenza, di visualizzarne il payload, ottenendo un suggerimento per la soluzione della challenge.

Inoltre, il payload inviato potrebbe essere tale da mandare in crash il servizio target, causando una penalita alla percentuale di **SLA** *(Service Level Agreement)*, ed interferendo potenzialmente con il lavoro del team sfidante (portando per esempio alla perdita di alcuni file). Si vuole dunque trovare un sistema di verifica che esegua completamente all'oscuro del team sfidante.

Il sistema di verifica tramite sistema esterno non copre nemmeno tutte le vulnerabilita. Banalmente, non tutti i servizi vulnerabili sono esposti in rete, ma sono accessibili solo in locale.

Riassumendo quindi, abbiamo identificato alcuni spunti per ideare un meccanismo di verifica migliore:

- Il traffico di rete non dovrebbe essere letto dal team sfidante al fine di ottenere suggerimenti per la soluzione.
- Se l'exploit manda in crash il servizio, questo non dovrebbe penalizzare il team, o piu in generale: il team sfidante non vede il servizio distruggersi.
- Dobbiamo gestire anche vulnerabilita non esposte in rete.

1.2.3 Verifica Dinamica tramite injection dei comandi di verifica

Questo meccanismo di verifica prevede l'uso di un **orcehstratore esterno** al sistema vulnerabile, che ha la facolta di eseguire comandi all'interno delle macchine vulnerabili che compongono il sistema. Dunque l'orchestratore esegue uno o piu comandi con il fine di verificare che il sistema sia resistente alla vulnerabilita.

Considerando il caso in cui una macchina `A` contenga un file binario vulnerabile ad un *buffer overflow*, l'orchestratore potrebbe eseguire ad esempio il comando `/path /to/service_logger payload` e verificare l'esito dell'esecuzione.

L'esecuzione di questi comandi e' possibile per esempio in ambienti containerizzati, sfruttando utility come `docker exec` o `incus exec`, che permettono di iniettare

comandi all'interno dei container.

Tuttavia: comandi come `pspy`, (che ascoltano attivamente le chiamate di sistema effettuate e gli eventi sul file system in `/proc`) possono intercettare l'esecuzione dei comandi dell'orchestratore e di mostrarli al team, suggerendo al team sfidante la soluzione. [2]

Dunque, dobbiamo ideare un meccanismo di verifica che deve sia completamente invisibile a questi meccanismi di intercettazione.

1.2.4 Clonazione del sistema in una rete separata

In base a quanto visto prima, in BlueAgent faremo cosi:

1. Si clonano tutte le macchine del sistema vulnerabile, in una rete separata, mantenendone la topologia originale.

- – La rete di verifica per il team k-esimo sara' chiamata verify-k

2. Gli sfidanti non devono poter instradare i pacchetti da e verso la rete verify-k.
3. Gli sfidanti non devono poter fare injection dei comandi nel sistema nella rete verify-k.
4. Si aspetta che il sistema si operativo, facendo dei controlli di readyness.
5. Si esegue una batteria di test sul sistema clonato. Questo test e' costituita da una serie di comandi da iniettare nel sistema clonato.
6. Per ogni comando eseguito si comparano `stdout`, `stderr` ed `exit_code` con il valore attesso.
7. Se il test ha avuto successo, allora BlueAgent risponde con lo stato `PATCHED`. (TODO: CONTROLLA CHE SIA CORRETTO!)

1.3 Cyber-Range: lo stato dell'arte

Secondo il NIST, un **Cyber-Range** (CR) e' definito nel seguente modo:

> "Un'ambiente interattivo, che simula la rete locale, i sistemi, i tool e le applicazioni proprietarie di un'azienda o di un'organizzazione". [3]

Dunque i CR giocano un ruolo fondamentale per formare ed addestrare i futuri professionisti del campo ICT.

Ilustreremo nelle sezioni successive alcuni Cyber Range esistenti e valuteremo la loro applicabilita al problema delle competizioni CTF gestite da BlueAgent e se esistono meccanismi di verifica alternativi validi per il nostro scopo.

[2] `https://github.com/DominicBreuker/pspy`

[3] `https://www.mdpi.com/1424-8220/20/24/7148` 1. Introduction

1.3.1 CR Alternativi: kCTF

kCTF[4] e' una piattaforma sviluppata da Google, basata su *Kubernetes* e *nsjail*.

> *Kubernetes*: e' un sistema open source per automatizzare il deploy, scalare e gestire applicazioni containerizzate.[5]
>
> *NSJail*: ossia, NameSpace Jail, e' un tool Linux per isolare i processi utilizzando varie feature del Kernel Linux come: namespaces, resource limits, seccomp-bpf syscall filters. L'obiettivo di NSJail e' quello di creare un'ambiente sicuro e isolato per eseguire le applicazioni. [6]

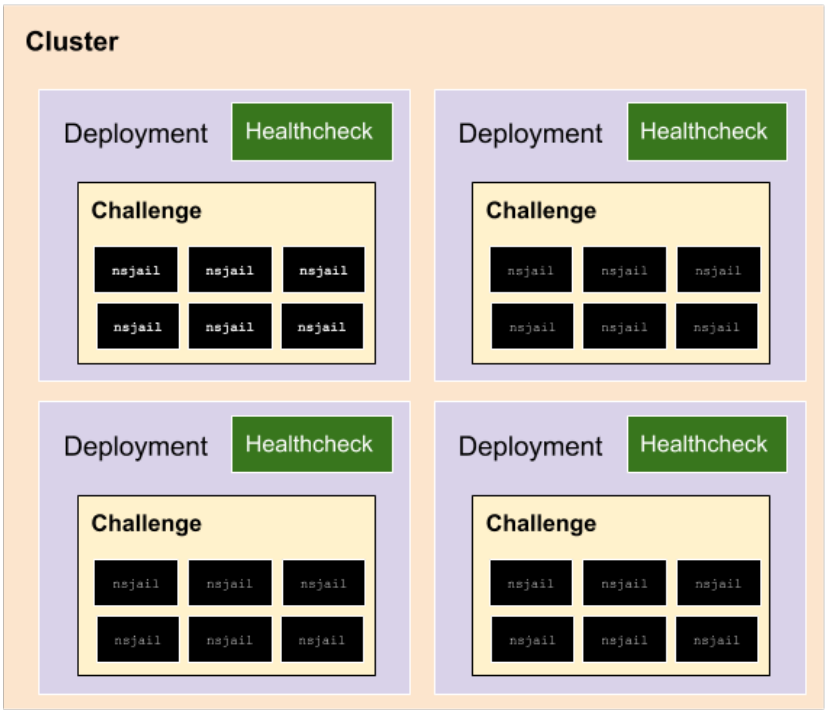


Figura 1.1: Kctf Architecture

Facendo riferimento all'immagine sopra, l'architettura di kCTF prevede:

- Un **cluster**, che contiene un'insieme di challenge
- Dei **deployment**, ognuno contiene due container (o meglio, un Pod che instanzia 2 container):

[4] `https://google.github.io/kctf/`
[5] `https://kubernetes.io/`
[6] `https://github.com/google/nsjail/`

- un container `healthcheck`: usato per determinare lo stato di salute dei servizi
- un container `challenge`: contiene i servizi su cui lavora lo sfidante.

Pod: in Kubernetes, e' la piu piccola unita che Kubernetes puo' eseguire. Un Pod, rappresenta uno o piu container in esecuzione su un singolo nodo.[7]

Deployment: in Kubernetes, un deployment e' un template che specifica come creare i container. I template guidano la creazione di **Pods** e la gestione sicura del ciclo di vita di un Pod, in particolare la gestione dei ReplicaSet.[7]

NSJail viene avviato su ogni container `challenge`, ed e' configurato per ascoltare su una porta TCP nota. Ad ogni connessione TCP in arrivo, NSJail esegue un fork restituendo allo sfidante un'ambiente separato (rispetto agli altri sfidanti) su cui lavorare. L'obiettivo di questa architettura e' di poter riutilizzare lo stesso container per piu sfidanti.

L'isolamento fornito da NSJail, si ottiene combinando varie tecniche:

- Creazione di nuovi namespace: l'ambiente fornito, esegue su un nuovo namespace, separando logicamente le risorse accessibili.
- Filtraggio delle chiamate di sistema tramite `seccomp-bpf`: questo sottosistema permette di uccidere i processi nell'ambiente se vengono eseguite chiamate di sistema vietate.
- Accounting e limitazione delle risorse con `cgroup`: questo sottosistema permette di monitorare e limitare le risorse utilizzabili nel nuovo ambiente.

NsJail puo' essere configurato in modo che ogni ambiente isolato, abbia a disposizione un'filesystem temporaneo che risiede in RAM, mentre il filesystem originale e' accessibile dall'ambiente sandbox in sola lettura. Questa configurazione e' necessaria poiche altrimenti gli sfidanti modificherebbero tutti lo stesso filesystem.

Un'altra configurazione di NSJail e' la seguente: che e' ideale per le challenge di tipo `pwn` (trovare e sfruttare vulnerabilita in file binari):

```
1 [...]
2 mount: [
3   {
4     src: "/chroot"
5     dst: "/"
6     is_bind: true
7   },
```

[7] `https://www.k8s.guide/workloads/pods-deployments/`

```
8     {
9       dst: "/tmp"
10      fstype: "tmpfs"
11      rw: true
12    },
13    {
14      dst: "/proc"
15      fstype: "proc"
16      rw: true
17    },
18    {
19      src: "/etc/resolv.conf"
20      dst: "/etc/resolv.conf"
21      is_bind: true
22    }
23  ]
```

1. La cartella `/chroot` che risiede nel container, diventa la radice nel nuovo ambiente.
2. Viene creato un nuovo `tmpfs`, in `/tmp`
3. Viene importata la configurazione DNS da `/etc/resolv.conf` dentro la nuova radice.

Tutta via, l'uso di NsJail per separare gli ambienti dei partecipanti, rende difficoltosa l'implementazione di challenge in cui si vuole dare la liberta ai partecipanti di installare pacchetti dentro al container, oppure di modificare file di configurazione che risiedono per esempio in `/etc` e di modificare direttamente servizi in esecuzione sul container. [8]

Queste limitazioni ci sono perche NsJail nell'ambiente sandbox non fornisce una propria istanza di `systemd` (o un qualsiasi gestore di servizi) e neanche l'infrastruttura necessaria per eseguiure comandi come `apt`.

NsJail porta con se altri problemi:

- Se dovesse cadere la connessione TCP allora tutto il contenuto del filesytem in RAM andrebbe perso.
- L'uso di `seccomp` per filtrare le chiamate di sistema potrebbe impedire l'esecuzione di alcuni programmi fondamentali per la challenge.

Queste limitazioni esposte, portano ad escludere kCTF come possibile soluzione al problema orignale esposto, suggerendo come astrazione migliore per l'ambiente di un partecipante l'uso di un **container** (Incus o Docker), piuttosto che una sandbox generata da NsJail.

[8] `https://github.com/google/kctf/blob/v1/docs/security-threat-model.md`

1.3.2 CR Alternativi: CTFBox

CTFBox e' un Cyber Range perfetto per organizzare CTF di tipo Attack/Defense, dove ogni team ha una macchina con uno o piu servizi vulnerabili da difendere, mentre si attaccano i servizi esposti sulle macchine avversarie per recuperare le flag. Durante il gioco, ogni team deve fare attenzione a non rompere i propri servizi, mantenendo un'alta percentuale di SLA. Una SLA bassa comporta una penalita' in termini di punteggio. [9]

Fa uso di **Docker** per gestire i servizi essenziali del gioco (e che costituiscono quindi CTFBox), mentre **Incus** viene utilizzato per gestire l'esecuzione delle macchine virtuali o dei container dei team.

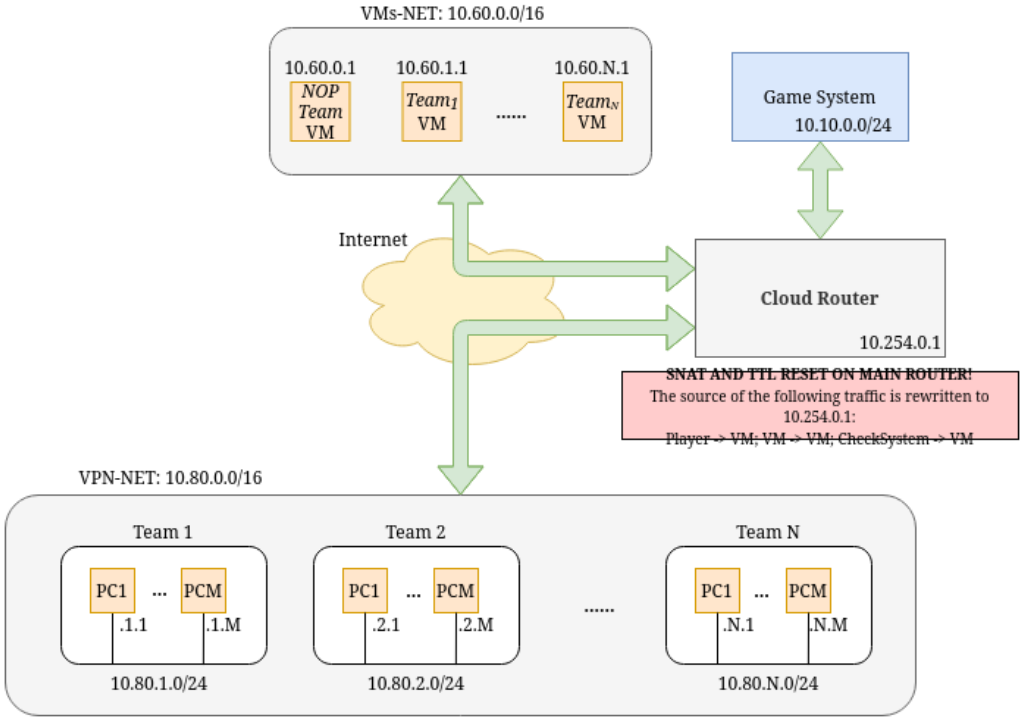


Figura 1.2: Ctfbox Architecture

Router: e' un container Docker, che gestisce le configurazioni della rete WireGuard, permettendo ai giocatori di connettersi alla rete di gioco. Controlla le regole di routing sulla rete, e governa la transizione della rete tra 3 stati diversi:

- `freeze`: in questo stato, nessuno puo' accedere alle macchine vulnerabili
- `lock`: in questo stato, ogni team puo' accedere solo alla propria macchina vulnerabile.
- `unlock`: in questo stato, la rete e' completamente aperta, i team possono attaccare le macchine avversarie.

[9] `https://ctfbox.domy.sh/`

Game System: e' un container Docker, che agisce da cervello centrale per la gestione del gioco.

- Gestisce lo stato della rete (freeze/lock/unlock), comunicando i cambiamenti al router.
- Mantiene il database delle flag, e calcola i punteggi per ogni team.
- Coordina i Checkers, ossia i worker che interrogano i servizi dei team ad ogni tick.

In CTFBox, ad ogni tick (ossia ogni round di gioco), le flag vengono rigenerate su ogni macchina, per ogni servizio vulnerabile, e registrate nel database. L'obiettivo di un team in una competizione Attack/Defense strutturata in questo modo e' di attaccare piu volte lo stesso servizio sulle macchine avversarie, per recuperare quante piu flag possibli, finche le difese avversarie non bloccano gli attacchi.

Checkers, ad ogni tick (o round) fanno due cose:

1. Interrogano tutti i servizi vulnerabili attivi, e determinano la percentuale di SLA per ogni team.
2. Installano le nuove flag sui servizi vulnerabili.

Ogni membro del team, ha la sua configurazione **WireGuard** per conettersi alla propria macchina e per raggiungere le macchine avversarie. Ad ogni giocatore che si connette viene assegnato un'indirizzo nella sottorete `10.80.team_id.0/24` mentre le macchine dei team si trovano nella sottorete `10.60.0.0/16`.

Dunque CTFBox si rileva essere un'ottimo sistema per gestire competizioni Attack/Defense e per regolare l'accesso alle macchine, ma la sua architettura non si applica in contesti in cui vorremmo solo assegnare ad ogni team un insieme di macchine vulnerabili il cui stato verra' poi verificato da un'oracolo.

1.3.3 CR Alternativi: CyTrONE

CyTrONE[10] e' un framework incentrato sul training che integra contemporaneamente sia una piattaforma educativa che un Cyber Range.

La seguente immagine mostra i componenti di CyTrONE:

- **CyLMS**: si occupa d icaricare il contenuto per il training su un Learning Management System come **Moodle**
- **CyRIS** (Cyber Range Instantiation System): si occupa di istanziare l'ambiente di training su cui gli studenti possono condurre opportuni test per avanzare nel training.

[10] `https://github.com/crond-jaist/cytrone`

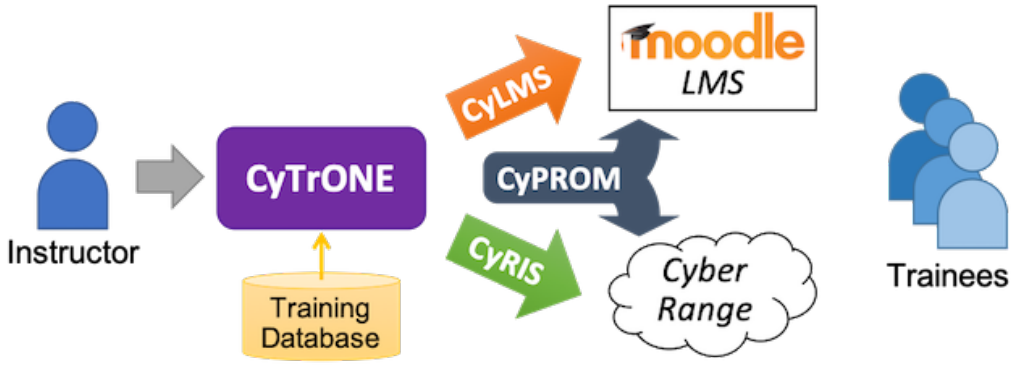


Figura 1.3: Cytrone Architecture

- **CyPROM**: si occupa di tracciare l'avanzamento controllando LMS ed il Cyber Range.

CyTrONE gestisce tre tipi di training[11]:

- Training *Attack-oriented*: si basa sul capire come riprodurre attacchi a vulnerabilita note.
- Training *Analysis-oriented*: si concentra sull'analisi dell'attacco, per capire come funziona e le sue conseguenze.
- Training **Defense-oriented**: si conentra sulle tecniche di patching delle vulnerabilita, al fine di proteggere il sistema.

Quindi l'esperienza utente si concentra sul fatto che gli studenti possono eseguire un training e visualizzare le risorse associate sul Moodle ed al tempo stesso accedere al Cyber Range associato.

CyTrONE si applica bene facendo le seguenti assunzioni:

1) *L'obiettivo di ciascuna attività di formazione può essere formulato come un problema, che può dipendere o meno da altri problemi della stessa lezione (es: trova l'indirizzo IP, trova la flag);* [11]

2) *Il sistema è in grado di stabilire automaticamente se il partecipante abbia risolto il problema, confrontando la risposta fornita con un valore noto* [11]

L'architettura di CyTrONE e' supportata da un *Training Database*, che si occupa di tenere in memoria le informazioni per i vari training, che vengono caricate su CyTrONE da una figura identificata come *Training Coordinator*.Le informazioni mantenute nel database sono per esempio: il training content, file e programmi, macchine virtuali, ecc... [11]

CyTrONE utilizza un file YAML per definire il contenuto di un training:

```
---
- training:
  - id: L1-EN
    title: |
      Investigate the security of a desktop
      computer
    overview: |
      Today is your first day on the job as
      a sysadmin. Your boss tells you that he
      suspects somebody tried to hack into
      your company's network, and asks you to
      investigate a possible cyber attack
      that may have happened when the system
      administrator was a guy called Daniel
      Craig. The boss sits you in front of
      the previous sysadmin's computer, and
      wishes you good luck.

      You glance at the machine and
      reluctantly get to work.
    level: 1

    questions:
    - id: L1-EN-001
      body: |
        The operating system and kernel
        release number can tell you about
        the possible vulnerabilities of a
        computer. Find out the full kernel
        release number of the machine (e.g.,
        3.4.5-6.7.8.abc.x86_64).
      answer: 3.10.0-514.21.1.el7.x86_64
      hints:
      - You can use the command uname to
        find out OS details.
      - $ uname -r
      - An alternative solution is to check
        the /proc/version file.
```

Figura 1.4: Cytrone Training

```
---
- host_settings:
  - id: host_1
    mgmt_addr: {{ host_mgmt_addr }}
    virbr_addr: {{ host_virbr_addr }}
    account: {{ host_account }}

- guest_settings:
  - id: desktop
    basevm_host: host_1
    basevm_config_file: /imgs/basevm_desktop.xml
    basevm_type: kvm
    basevm_name: basevm_desktop
    tasks:
    - add_account:
      - account: daniel
        passwd: daniel_passwd
        full_name: Daniel Craig
    - install_package:
      - package_manager: yum
        name: wireshark
    - emulate_attack:
      - attack_type: ssh_attack
        target_account: daniel
        attempt_number: 123
        attack_time: 20170123
    - emulate_malware:
      - name: DAEMON
        cpu_utilization: 40
        mode: dummy_calculation

- clone_settings:
  - range_id: {{ clone_range_id }}
    hosts:
    - host_id: host_1
      instance_number: {{ clone_instance_number }}
      guests:
      - guest_id: desktop
        number 1
        entry_point: yes
      topology:
      - type: custom
        networks:
        - name: office
          members: desktop.eth0
```

Figura 1.5: Cytrone Cyberrange

E similmente, si usa un file YAML per definire la specifica del Cyber Range:

A partire dalla configurazione YAML del Cyber Range, il sotto-sistema CyRIS si occupa di istanziarlo. Successivamente, in caso di un training **Defense-oriented** l'utente richiede l'attacco associato alla vulnearbilita all'interno del sistema e ne viene registrato l'esisto.

Tutta via, sebbene il sistema sia ben ingegnerizzato soffre di alcuni problemi:

1. Il deploy delle vulnerabilita avviene su macchine virtuali, che possono costituire un alto overhead in termini di risorse, ed tempi di approvvigionamento lunghi per l'infrastruttura.
2. Il payload viene inviato remotamente dal `cyberattack controller` (che non fa altro che eseguire `Metasploit`). Tutta via il traffico di rete puo' essere intercettato dallo studente, ottenendo cosi suggerimenti per la soluzione.
3. L'esecuzione del payload puo scatenare la produzione di file log accessibili allo studente.
4. L'esecuzione del payload potrebbe mandare in crash il sistema del partecipante.
5. Necessita di un LMS (per design) per tenere traccia del training progress.

1.3.3.1 Valutazione delle Performance di CyTrONE

Le performance di CyTrONE per il deploy dell'infrastruttura sono state valutate usando il testbed StarBED [12], usando 30 host fisici e 600 macchine virtuali. Gli host fisici usati per misurare le performance sono dei *server rack Cisco UCS C200 M2 con Intel Xeon E5504 2.0 GHz a 4 core e 72 GB di memoria*

Sono stati valutati 2 scenari:

- *Level 1*: un training base che include tra gli argomenti: log, configurazioni di sistema, sniffing della rete e scansione delle vulnerabilita. Richiede un'unica macchina Virtuale per fare il deploy dello scenario
- *Level 2*: un training di difficolta media su topic come il network discovery, penetration testing e password cracking. Richiede due macchine virtuali per il deploy dello scenario.

Il processo di deploy del training, per una sola macchina virtuale, in CyTrONE avviene in 3 fasi distinte:

1. **Preparazione**: Su un nodo denominato *master* viene configurata (per esempio, accesso tramite SSH) ed avviata la macchina virtuale. T_p descrive il suo tempo di esecuzione

[11] `https://www.sciencedirect.com/science/article/pii/S0167404818306527`
[12] `https://www.sciencedirect.com/science/article/pii/S0167404818306527#bib0015`

2. **Installazione**: Sulla macchina nel nodo master, vengono installati contenuti aggiuntivi necessari per il traninig, sulla macchina virtuale target. T_i descrive il suo tempo di esecuzione.
3. **Clonazione**: Copia tramite `parallel-scp` dell'immagine della macchina sui vari nodi della rete, garantendo al tempo stesso che password e credenziali siano distinte per ogni immagine. T_c descrive il suo tempo di esecuzione

   E' dunque importante notare che per il *Level 1*, bisogna clonare con `parallel-scp` una sola immagine per nodo, mentre per il *Level 2* bisogna clonare 2 immagini per nodo.

Sia $T = T_p + T_i + T_c$, il grafico descrive come si comporta T al variare del numero di host su cui fare il deploy delle macchine.

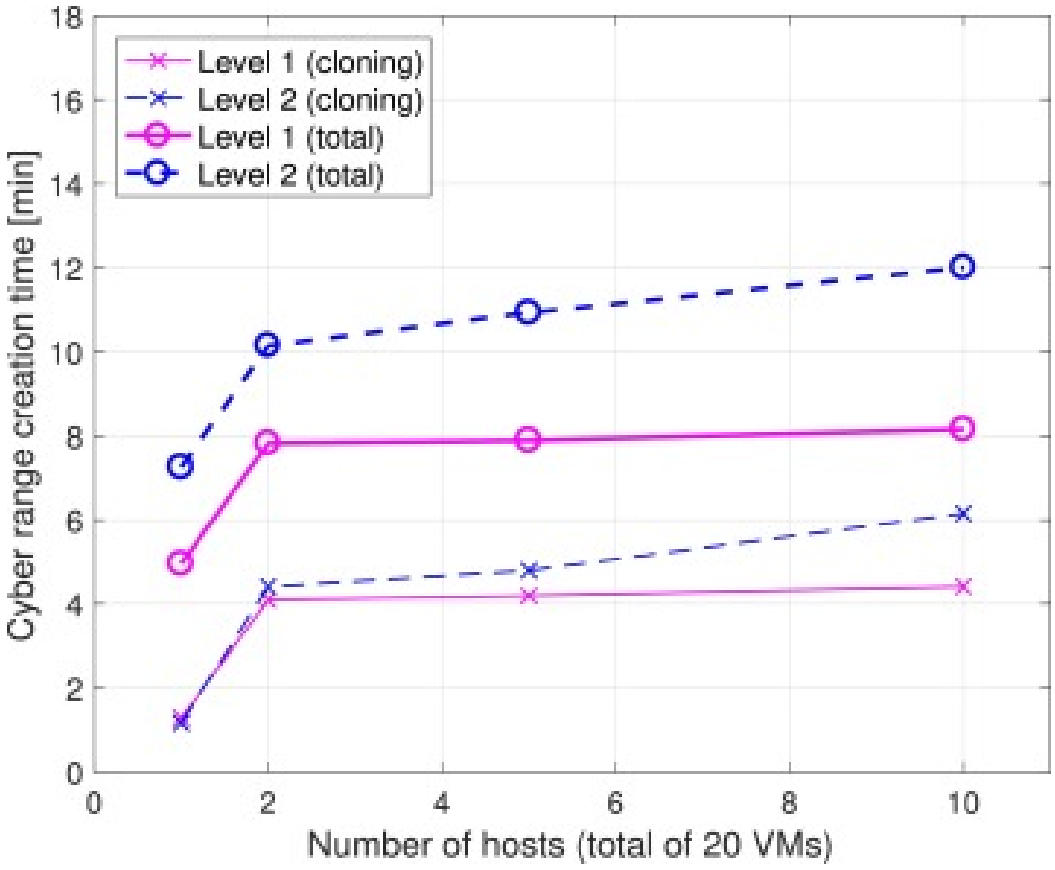


Figura 1.6: Cytrone Performance

- Nei casi dove abbiamo solo 1 host, allora non c'e' bisogno di clonare l'immagine, basta solo un host che esegue le 20 macchine virtuali, ossia il master. Dunque il tempo di clonazione e' sotto i *due minuti*, mentre T per lo scenario *Level 2* si attesta sotto gli 8 minuti.

- All'aumentare del numero n degli host, con il bisogno di clonare le macchine su piu nodi, il tempo di approvvigionamento totale tende a peggiorare, poiche bisogna trasferire l'immagine su piu nodi, sebbene ogni nodo gestisca solo $20/n$ macchine virtuali.

Invece, per gli esperimenti su larga scala dove si vuole effettuare il deploy di 20 macchine per ogni host su centinaia di host il tempo T peggiora esponenzialmente. Nella figura sborgz viene analizzato al variare di del numero di host fino a 600 cosa succede se ogni nodo deve eseguire 20 macchine, con 20 immagini differenti.

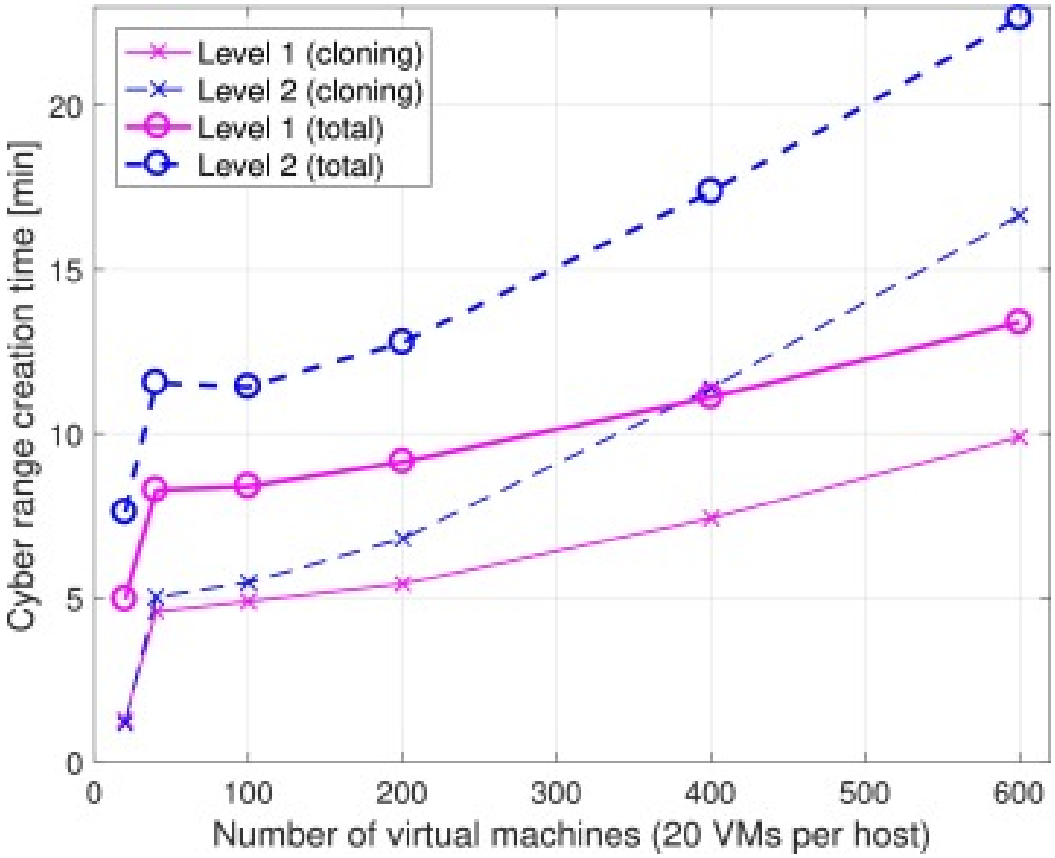


Figura 1.7: Cytrone Largescaleperformance

L'analisi delle performance permette di concludere che il sistema non e' ammissibile per organizzare competizioni che non hanno a disposizione macchine performanti come quelle utilizzate nei test mostrati prima. A priori dall'hardware a disposizione, l'uso di macchine virtuali costituisce un grande overhead in termini di memoria usata e di CPU allocata nel momento in cui la stessa macchina deve essere replicata per piu partecipanti.

Con il nostro lavoro di tesi, si vuole cercare di abbattere il tempo T, passando dall'ordine dei minuti, all'ordine dei secondi su hardware economico.

1.3.4 CR Alternativi: CyberRangeCZ

CyberRangeCZ e' un fork di **KYPO**. E' una piattaforma basata su OpenStack per fornire un'ambiente cloud su cui fare il deploy delle macchine. Utilizza tecnologie moderne come container, IaC (infrastructure as code) ed i microservizi.

Macchine virtuali, reti, e training sono definiti utilizzando JSON e YAML. Questi file vengono passati ad Ansible per descrivere il contenuti della macchina e ad Packer per costruire le immagini delle macchine virtuali.

Fa uso massivo della virtualizzazione per ricreare fedelmente reti complesse ed interi sistmi operativi.

La piattaforma si concentra per soddisfare due casi d'uso principali:

- emulare intere infrastrutture di computer in ambienti controllati
- eseguire più sessioni di training contemporaneamente, gestendo centinaia di ambienti virtuali per centinaia o migliaia di partecipanti.

La piattaforma CyberRangeCZ si occupa di creare e approvvigionare un'ambiente virtuale emulato costituito da Sandbox e Pool.

> **Sandbox**: *"È un ambiente di test isolato con reti virtuali che consente agli utenti di connettersi alle macchine virtuali (VM) e comunicare con il resto della rete, la macchina host e le altre VM. Sulle VM, l'utente può eseguire programmi, utilizzare servizi di rete o monitorare il traffico di rete, il tutto senza influire sull'infrastruttura esterna. La sandbox viene eseguita all'interno del cloud ed è accessibile da remoto."* [13]
>
> **Pool**: *"Un gruppo di sandbox cloud create a partire dalla medesima sandbox definition."* [13]

All'interno di una sandbox e' possibile fornire una **Topology Definition**, un file che decsrive la struttura dell'ambiente (host, router, reti, etc.) mentre il **Sandbox Provisioning** si occupa di customizzare le instanze nella topologia.

La creazione di una Sandbox si divide in 3 fasi:

- **Allocazione**: creazione della sandbox (=macchine virtuali) dentro al cloud.
- **Networking**: setup della rete e distirbuzione delle chiavi di accesso dentro le macchine.
- **Provisioning**: customizzazione delle macchine virtuali nella sandbox.

  La creazione della Pool precede necessariamente la creazione delle Sandbox.

[13] `https://docs.platform.cyberrange.cz/basic-concepts/terminology/#emulated-virtual-environment`

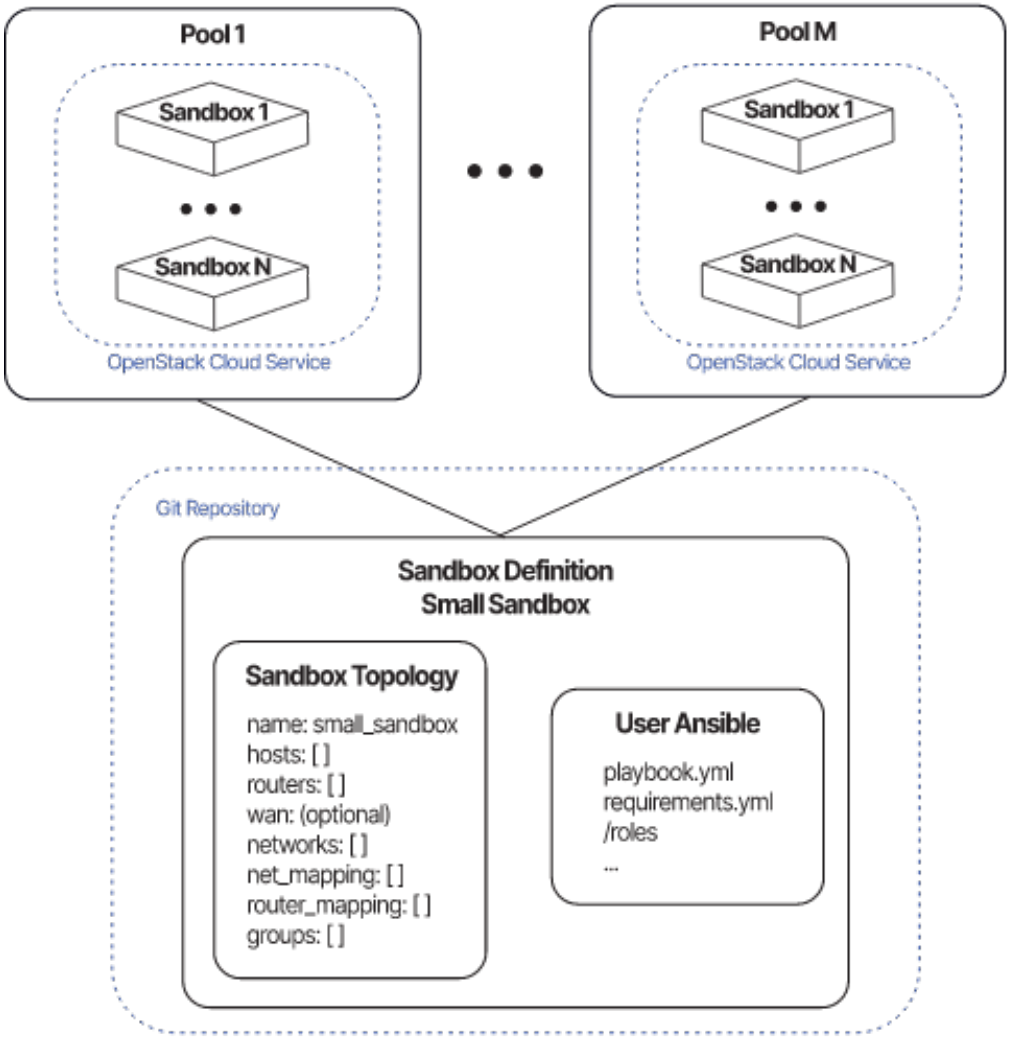


Figura 1.8: Kypo Basic Elements Sandboxes

A partire dalla Topology Definition, viene prodotta la **Topology Instance** utilizzando `Terraform` (che crea l'infrastruttura virtuale). L'accesso alla topologia e' regolato da un MAN (Management Access Node), ossia l'unico nodo connesso all'esterno della rete della Topology Instance.

Di seguito, nell'immagine sburgoz, viene mostrata l'architettura generale di una Topology Instance.

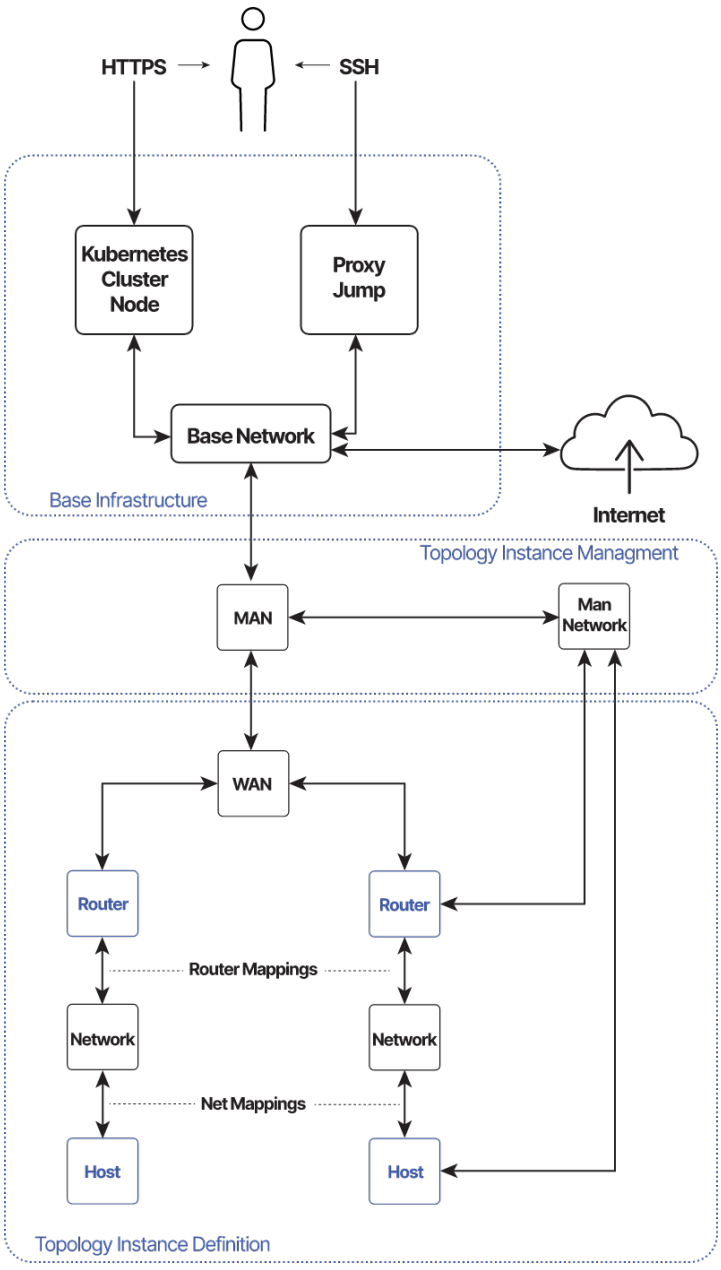


Figura 1.9: Kypo Topology Instance

L'accesso alle macchine dentro la Sandbox e' regolato tramite SSH, oppure da una console web accessibile via browser.

Gestire Sandobx, ossia intere Topology Instance e' costoso, infatti i requisiti sono

i seguenti:

- una macchina virtuale con almeno *2 core CPU, 16 GB di RAM e 40 GB di archiviazione*
- un server dotato di almeno *16 core CPU, 128 GB di RAM, 2 TB di archiviazione e almeno due interfacce di rete.*

Requisiti cosi alti sono necessari per:

- gestire l'infrastruttura OpenStack, i microservizzi, e Kubernetes
- gestire l'esecuzione di centinaia di macchine virtuali e garantire la scalabilita al crescere degli utenti. Si consideri che uno scenario base potrebbe per esempio richiedere 4 CPU virtuali, 4 GB di RAM e 41 GB di disco. Per 30 partecipanti si arriva a consumare gia 1.2TB di spazio sul disco.

Quindi CyberRangeCZ e' un software complesso e flessibile che si applica a molti scenari, tutta via, non si applica al nostro problema orignale poiche:

- Creare un'intera topologia richiede l'uso di piu tool tra cui Ansible, alzando all'ordine dei minuti il tempo per il provisioning della Sandbox.
- Non e' previsto un meccanismo di verifica dello stato delle macchine come discusso in **1.2**
- Inoltre, i requisiti di minimi sono eccessivi e si potrebbe pensare anche di migliorare questo aspetto.

1.3.5 CR Alternativi: CyberPatrion

E' un programma di educazione nazionale per il K-12 creato dalla *Air & Space Forces Association*. I team assumono giocano il ruolo di professionisti di cybersecuirity che hanno l'obiettivo di proteggere vari sistemi entro un limite di tempo, analizzando le macchine virtuali, identificandone le vulnerabilita e mitigandole.

Il CyberPatriot Scoring System (CSS) tiene traccia della progressione del team. Un demone locale esegue controlli periodici sulla macchina per determinarne lo stato, mentre un server centrale registra i punteggi comunicando con i demoni delle macchine.

Questo sistema tutta via e' **closed-source** e solo l'**AFA** puo' utilizzarlo.

Capitolo 2

BlueAgent

2.1 2. Introduzione a BlueAgent

Si vuole presentare dunque il nostro CyberRange: **BlueAgent**. Un servizio *HTTP* che implementa delle *API* per la gestione e la verifica delle macchine a partire da definizioni *IaC*. Queste API potranno poi essere sfruttate ed implementate da qualsiasi applicativo per gestire le challenge, come per esempio un plugin per CTFd che vedremo in seguito.

BlueAgent e' basato **FastAPI**, per gestire in modo semplice ed efficiente le richieste HTTP.

L'host che su cui viene eseguito BlueAgent deve mettere a disposizione dell'architettura le seguenti componenti:

- **Debian 13 ed il Kernel Linux**: per la gestione delle interfacce di rete
- – `nftables`: per gestire le regole di inoltro nel Kernel
- **ZFS**: filesystem che supporta il meccanismo `COW`.
- **OVN ed Open vSwitch**: per gestire la rete di verifica dei container.
- **distrobuilder**: per generare le immagini dei container in modo dichiarativo.
- **WireGuard**: per fornire una VPN come punto d'accesso ai partecipanti
- **Incus**: per la gestione dei container.

Il codice in Python che implementa BlueAgent usa i seguenti moduli:

- **uv**: per gestire in modo semplice ed efficace il `venv` ed i pacchetti richiesti da BlueAgent
- **pydantic**: per modellare e validare le strutture dati utilizzate frequentemente.
- **filelock**: per implementare il lock delle risorse in caso di accesso concorrente.
- **aiosqlite**: per interfacciarsi con il database.

Per comprendere passo-passo cosa permette di fare il sistema e come lo fa sono elencati gli use cases ed i requisiti di sistema.

2.2 Attori

L'attore e' un'entita logica esterna al sistema che interagisce direttamente con esso, tramite l'invio di messaggi.

2.2.1 CTF Manager (attore)

E' un sistema software esterno a BlueAgent che si occupa di gestire la competizione (punteggi, team, utenti, configurazioni, gara). Comunica con BlueAgent tramite API REST per gestire l'intero ciclo di vita delle macchine dei team, ed agisce come entita autenticata (tramite l'utilizzo di un token) per BlueAgent.

Richiede per conto dei Giocatori il deploy delle challenge, la verifica dei container e le configurazioni per accedere alla rete tramite VPN.

Per conto dell'autore della sfida permette la gestione dei file di configurazione delle challenge. Permette invece all'admin di gestire i container attivi, i team, e altri aspetti della gara.

Piu avanti, verra presentato un plugin per CTFd, che agisce come CTF Manager verso BlueAgent.

Dunque CTF Manager e' l'unico attore per BlueAgent

Rendere il CTF Manager esterno ed indipendente da BlueAgent nella sua architettura, permette di non vincolare l'uso di BlueAgent con CTFd per esempio.

2.3 CTF Manager (sistema)

Ora vogliamo definire meglio il ruolo che hanno il Admin, Giocatore ed Autore della sfida. Chiariremo subito che il Giocatore, l'Autore della sfida e l'Amminstratore della competizione non sono attori che dialogano con BlueAgent.

2.3.1 Giocatore (attore esterno per CTF Manager)

E' un'entita che agisce in **team** con altri giocatori. Ha accesso alle macchine delle challenge istanziate da BlueAgent, appartenenti al suo team. Non comunica direttamente con BlueAgent ma piuttosto si interfaccia con il **CTF Manager** per gestire la progressione nella competizione.

2.3.2 Autore della sfida (attore esterno per CTF Manager)

Si occupa di creare i file di configurazione per una o piu challenge e di inviare tramite CTF Manager le immagini dei container definite tramite configurazione distrobuilder. Dunque non dialoga direttamente con BlueAgent

2.3.3 Amministratore della competizione (attore esterno per CTF Manager)

Per aspetti legati alla gestione della competizione CTF si interfaccia direttamente con il CTF Manager. Se ha bisogno di interagire con configurazioni di team e container, CTF Manager cattura le richieste dell'amministratore e le inoltra al blueagent.

2.4 Use Cases per CTF Manager

2.4.1 `UC-01`: Deploy dell'infrastruttura di rete per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Effettuare il deploy delle interfacce di rete necessarie al team k per eseguire i container e consentire l'accesso VPN ai partecipanti.
- **Precondizione**: BlueAgent e' attivo ed e' raggiungibile via API REST.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams` fornendo l'oggetto pydantic `TeamCreateReq`, specificando l'identificativo k del team (`team_k`) ed il relativo nome (`name`).
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent verifica se l'infrastruttura per il team k esiste gia all'interno del database:
    1. Se il team e' gia registrato, risponde `HTTP 200 OK`.
    2. Altrimenti, l'infrastruttura di rete viene creata da zero:
        1. Viene creato il Linux bridge `br-team-k` per il team k con IP per il gateway `10.k.1.1/24`.
        2. Viene creata l'interfaccia WireGuard `wg-k` per il team k. La porta UDP e' `51820+k`, l'IP per il gateway e' `10.k.0.1/24` e la `MTU` e' impostata a 1420.
        3. Viene registrato il team all'interno del database SQLite.
        4. BlueAgent rilascia il lock per il team k.
        5. BlueAgent risponde con stato `HTTP 201 Created` confermando l'avvenuta creazione.

Se avviene un errore irrimediabile durante la creazione, il bridge e l'interfaccia wireguard create vengono rilasciate.

2.4.2 UC-02: Deploy della challenge per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Effettuare il deploy dei container necessari alla challenge da allestire per il team k richiedente.
- **Precondizione**: UC-01. E' stato eseguito con successo il deploy dell'infrastruttura di rete per il team k e le immagini dei container richieste sono presenti nel sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/machines` specificando `team_k` nel path e l'oggetto pydantic `ChallengeConfig` nel body.
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent valida le precondizioni.
4. Per ogni container specificato nella configurazione:
    1. Richiede al `ContainerEngine` (Incus) la creazione dell'istanza a partire dall'immagine indicata.
    2. Collega l'interfaccia virtuale `eth0` dell'istanza al bridge del team `br-team-k`.
    3. Inietta nel filesystem dell'istanza la configurazione di rete statica (`systemd-networkd` in `/etc/systemd/network/10-eth0.network` e `/etc/network/interfaces`)
    4. Avvia l'istanza del container ed applica immediatamente a runtime l'indirizzo IP statico.
    5. Registra il container nel database SQLite associandolo a `team_k` e `challenge_id`.
5. BlueAgent restituisce la risposta con stato `HTTP 201 Created` (`DeployResponse`).

Se si verifica un errore durante l'avvio o la configurazione di uno o piu container, BlueAgent: arresta ed elimina tutte le istanze create fino a quel momento per la challenge, pulisce i relativi record nel database, rilascia il lock e solleva `DeploymentFailedError`.

2.4.3 UC-03: Verifica della challenge per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Verificare l'integrita dei servizi e l'efficacia delle patch applicate dal team mediante l'esecuzione dei comandi specificati nella configurazione della challenge. Si vuole prevenire Side-Channel Leaks ed evitare alterazioni dell'ambiente di produzione del team k tramite la clonazione CoW dell'infrastruttura di produzione su una dark sandbox OVN.

- **Precondizioni**: `UC-02`. I container relativi alla challenge specificata per il team k sono attivi.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/verificator` trasmettendo l'oggetto pydantic `ChallengeConfig` della challenge da verificare.
2. BlueAgent richiede ed acquisisce il lock esclusivo per la verifica del team k. Se una verifica per lo stesso team e' gia in corso, risponde con `HTTP 409 Conflict`.
3. Altrimenti, BlueAgent inizializza il context manager asincrono `OVNVerifier`:
    1. Crea uno switch virtuale OVN denominato `verify-k`
    2. Esegue la clonazione sfruttando il meccanismo CoW a livello di blocchi di storage ZFS di tutti i container della challenge del team k.
    3. Riconfigura l'interfaccia `eth0` dei cloni agganciandola allo switch dark OVN appena creato.
    4. Avvia i cloni sandbox e configura al loro interno lo stesso indirizzo IP statico di produzione `10.k.1.{ip_suffix}/24`.
    5. Se specificato nella configurazione, avvia un container di verifica.
4. Esecuzione dei controlli di verifica (`readiness_commands`): viene eseguito un ciclo di tentativi finche' il comando non ha successo, oppure fino allo scadere del timeout.
5. Esegue i comandi di verifica (`verify_commands`) all'interno del container specificato.
6. Teardown atomico delle risorse garantito dal context manager: vengono eliminati i container, e lo switch OVN
7. BlueAgent rilascia il lock per il team k.
8. BlueAgent risponde con stato `HTTP 200 OK` restituendo l'oggetto pydantic `VerifyResultResp` che comunica l'esito della verifica.

Se si verifica un'errore le risorse utilizzate vengono rilasciate e viene comunicato l'errore.

2.4.4 `UC-04`: Deploy di un unico container per il team k [TODODOD]

- **Attore**: CTF Manager
- **Obiettivo**: Avviare un singolo container specifico per il team k, a partire dalla definizione della challenge che lo contiene.
- **Precondizioni**: `UC-01`. E' stato eseguito con successo il deploy dell'infrastruttura di rete per il team k. L'immagine del container richiesta e' presente nel catalogo in stato `ready`.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/machines` indicando l'ID del team nel path e passando la configurazione `ChallengeConfig` contenente la specifica della challenge che descrive il container da avviare.
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent verifica che il team k esista e che il bridge `br-team-k` sia attivo.
4. BlueAgent verifica che l'immagine richiesta sia disponibile nel catalogo e pronta (`ready`).
5. BlueAgent invoca il `ContainerEngine` (Incus) ed avvia il container.
6. BlueAgent rilascia il lock per il team k.
7. BlueAgent risponde con stato `HTTP 201 Created` confermando il deploy del singolo container.

In caso di fallimento durante l'avvio, BlueAgent esegue il rollback eliminando l'istanza creata, rilascia il lock e solleva `DeploymentFailedError` (restituendo un errore HTTP).

2.4.5 `UC-05`: Dismissione e rilascio di un unico container per il team k [TODO!!!@#@!#]

- **Attore**: CTF Manager
- **Obiettivo**: Arrestare ed eliminare un singolo container attivo per il team k.
- **Precondizioni**: Il container specificato appartiene al team k ed e' presente nel sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /teams/{team_k}/machines/{name}` specificando il nome del container da eliminare
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent individua l'istanza del container sul runtime (`ContainerEngine`) e sul database:
    1. Arresta l'istanza del container (`stop_instance`).
    2. Elimina l'istanza dal runtime Incus (`delete_instance`), scollegandola dal bridge `br-team-k` e rilasciando le risorse ZFS.
    3. Rimuove il record associato al container dalla tabella del database SQLite.
4. BlueAgent rilascia il lock per il team k.
5. BlueAgent risponde con stato `HTTP 204 No Content`.

In caso di errore durante la rimozione del container, BlueAgent rilascia il lock e solleva `TeardownError` rispondendo con errore `5XX`.

2.4.6 `UC-06`: Dismissione e rilascio di una challenge per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Rimuovere tutti i container associati a una determinata challenge per il team k.
- **Precondizioni**: I container associati alla challenge specificata sono attivi per il team k.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /teams/{team_k}/machines` specificando nel payload l'oggetto `ChallengeConfig` che definisce i container appartenenti alla challenge da dismettere.
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. Per ciascun container specificato nella configurazione della challenge:
    1. Arresta l'istanza tramite `ContainerEngine`.
    2. Elimina l'istanza dal runtime Incus liberando le risorse di storage ZFS.
    3. Cancella il record del container dal database SQLite.
4. BlueAgent rilascia il lock per il team k.
5. BlueAgent risponde con stato `HTTP 204 No Content`.

In caso di problemi durante il teardown di uno o piu container, BlueAgent rilascia il lock e restituisce un errore HTTP con i dettagli dell'anomalia.

2.4.7 `UC-07`: Dismissione e rilascio di un intero team k

- **Attore**: CTF Manager
- **Obiettivo**: Rimuovere completamente l'infrastruttura e tutte le risorse associate al team k.
- **Precondizioni**: Il team k e' registrato nel sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /teams/{team_k}` specificando l'identificativo `team_k`.
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent esegue la procedura di dismissione completa (`TeamService.teardown_team`):
    1. Arresta ed elimina tutti i container ancora attivi per il team k

2. Esegue il teardown dell'interfaccia WireGuard `wg-k` e le chiavi crittografiche associate ai Peer.
3. Elimina la regola per il routing da `nftables` e rimuove il bridge `br-team-k`
4. Cancella a cascata nel database SQLite il record del team k e tutti i record dei relativi membri e peer WireGuard registrati.

4. BlueAgent rilascia il lock per il team k.
5. BlueAgent risponde con stato `HTTP 204 No Content`.

L'operazione e' idempotente: se il team non esiste o e' gia stato dismesso, l'operazione completa con successo senza sollevare errori non gestiti.

2.4.8 UC-08: Compilazione e Import di un'immagine per container

- **Attore**: CTF Manager
- **Obiettivo**: Inviare la specifica dichiarativa di un'immagine in formato Distrobuilder YAML ed avviare la compilazione asincrona e l'importazione automatica nel pool di storage di Incus.
- **Precondizione**: BlueAgent e' attivo ed il compilatore `distrobuilder` e' installato sull'host.

Flusso delle azioni:

1. Il CTF Manager invia una richiesta `POST /images` inviando una form multipart con il file di definizione YAML di Distrobuilder (`definition`) e opzionalmente un alias univoco (`alias`).
2. BlueAgent acquisisce il lock globale per l'immagine associata all'alias (`locks.lock_global(f"image:{alias}")`).
3. BlueAgent valida la specifica YAML e ne calcola l'hash crittografico SHA-256.
4. BlueAgent verifica lo stato dell'immagine nel database SQLite:
    1. Se l'immagine con lo stesso alias o hash esiste gia, rilascia il lock e risponde immediatamente con `HTTP 200 OK` (idempotenza).
    2. Altrimenti, registra un nuovo record nel database e rilascia il lock.
5. BlueAgent avvia la compilazione in background tramite il worker asincrono (`ImageBuilder.build_image`):
    1. Individua il tarball unificato generato (`*.tar.xz` o `*.tar.gz`) e lo importa nel pool storage di Incus
    2. Assegna l'alias all'immagine importata e aggiorna il record nel database.
6. BlueAgent restituisce immediatamente al CTF Manager lo stato `HTTP 202 Accepted` attestando la presa in carico della richiesta in background.

2.4.9 `UC-09`: Dismissione di un'immagine per container [TP-DOAODODSA]

- **Attore**: CTF Manager
- **Obiettivo**: Rimuovere un'immagine non piu necessaria dal catalogo di BlueAgent e dallo storage di Incus.
- **Precondizioni**: L'immagine identificata dall'alias esiste nel sistema e non e' attualmente utilizzata da alcun container attivo.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /images/{alias}` specificando l'alias dell'immagine da eliminare.
2. BlueAgent acquisisce il lock globale per l'immagine.
3. BlueAgent verifica lo stato e i vincoli nel database SQLite:
    1. Verifica che l'immagine sia registrata a catalogo (altrimenti solleva `ImageNotFoundError` e risponde `HTTP 404 Not Found`).
    2. Verifica tramite vincoli di foreign key se vi siano container attivi che referenziano l'immagine:
        - Se l'immagine e' in uso da container attivi, BlueAgent rifiuta la richiesta sollevando `ImageInUseConflictError` e rispondendo con `HTTP 409 Conflict`.
4. Se l'immagine non e' utilizzata:
    1. Richiede al runtime Incus l'eliminazione dell'alias e la cancellazione dell'immagine dallo storage pool.
    2. Elimina il record dell'immagine dal database SQLite.
5. BlueAgent rilascia il lock per l'immagine.
6. BlueAgent risponde con stato `HTTP 204 No Content`.

2.4.10 `UC-10`: Ispezione dei Peer WireGuard

- **Attore**: CTF Manager
- **Obiettivo**: Monitorare lo stato della connettività VPN, le statistiche di traffico (byte ricevuti e trasmessi), gli handshake e lo stato online dei membri/giocatori del team k.
- **Precondizioni**: Il team k e' registrato e la relativa interfaccia WireGuard `wg-k` e' attiva sul sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `GET /teams/{team_k}/peers` specificando l'identificativo k del team.
2. BlueAgent interroga il kernel Linux eseguendo il comando `wg show wg-k dump` per estrarre le statistiche live dei peer (chiave pubblica, endpoint, ultimo handshake, contatori rx/tx).
3. BlueAgent interroga il database SQLite per recuperare i record dei membri registrati per il team k (`member_id`, `name`, `client_ip`, `public_key`, `ctfd_user_id`).
4. BlueAgent risponde con stato `HTTP 200 OK` restituendo la lista di `PeerStats`.

2.4.11 `UC-11`: Ispezione delle macchine in esecuzione

- **Attore**: CTF Manager
- **Obiettivo**: Ottenere l'elenco e lo stato di tutti i container attualmente istanziati per il team k.
- **Precondizioni**: Il team k e' registrato nel sistema.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `GET /teams/{team_k}/machines` specificando l'identificativo k del team.
2. BlueAgent interroga il database SQLite per recuperare i container registrati per il team k.
3. BlueAgent interroga il `ContainerEngine` (Incus) per verificare lo stato effettivo di ciascuna istanza nel runtime (`RUNNING`, `STOPPED`, ecc.).
4. BlueAgent risponde con stato `HTTP 200 OK` restituendo la lista dei container attivi.

2.4.12 `UC-12`: Healthcheck

- **Attore**: CTF Manager
- **Obiettivo**: Verificare la disponibilita operativa del servizio BlueAgent, la corretta raggiungibilita delle API REST e lo stato di salute generale del runtime.
- **Precondizioni**: Nessuna (l'endpoint e' pubblico e non richiede autenticazione).

Flusso delle azioni:

1. Il CTF Manager (o un sistema di monitoraggio esterno) invia la richiesta `GET /health`.
2. BlueAgent elabora la richiesta verificando la reattivita del server applicativo FastAPI.
3. BlueAgent risponde con stato `HTTP 200 OK` restituendo il payload JSON `{"status": "ok"}`.

2.4.13 UC-13: Registrazione e configurazione WireGuard per un membro del team k

- **Attore**: CTF Manager
- **Obiettivo**: Registrare un giocatore come membro del team k, allocare dinamicamente un indirizzo IP nella subnet VPN `10.k.0.0/24`, generare le chiavi crittografiche WireGuard (o registrare quelle fornite) e restituire il file di configurazione client `.conf`.
- **Precondizioni**: BlueAgent e' attivo. Se l'infrastruttura di rete del team k non e' ancora attiva, viene allestita contestualmente (lazy provisioning).

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `POST /teams/{team_k}/members` fornendo l'oggetto pydantic `TeamMemberCreateReq` (`name`, `ctfd_user_id`, opzionale `public_key`, opzionale `team_name`).
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent verifica se l'infrastruttura del team k e' attiva; se assente, esegue il provisioning trasparente dell'interfaccia `wg-k` e del bridge `br-team-k` (`UC-01`).
4. BlueAgent verifica se il membro e' gia registrato nel database per il team k (identificato da `ctfd-{team_k}-{ctfd_user_id}`):
    1. Se il membro esiste gia (es. download ripetuto della configurazione), BlueAgent fornisce la configurazione rispondendo con `HTTP 200 OK`.
    2. Se il membro e' nuovo:
        1. Alloca un indirizzo IP VPN libero per il team. Se lo spazio di indirizzamento e' esaurito (>253 membri), solleva `TeamAddressSpaceExhaustedError`.
        2. Se non fornita, genera una nuova coppia di chiavi asimmetriche.
        3. Registra il nuovo peer a livello kernel sull'interfaccia `wg-k`.
        4. Salva il record del membro nel database SQLite.
5. BlueAgent rilascia il lock per il team k.
6. BlueAgent risponde con stato `HTTP 201 Created` restituendo l'oggetto `TeamMemberWG` contenente il testo della configurazione WireGuard generata (`config_text`).

2.4.14 UC-14: Revoca di un membro WireGuard per il team k

- **Attore**: CTF Manager
- **Obiettivo**: Rimuovere un membro registrato dal team k, eliminando il relativo peer dall'interfaccia WireGuard `wg-k` e liberando l'indirizzo IP VPN allocato per futuri riutilizzi.

- **Precondizioni**: Il membro specificato e' registrato nel team k.

Flusso delle azioni:

1. Il CTF Manager invia la richiesta `DELETE /teams/{team_k}/members/{member_id}` specificando l'identificativo del team e del membro.
2. BlueAgent richiede ed acquisisce il lock per il team k.
3. BlueAgent individua il membro nel database SQLite:
    1. Se non trovato, solleva `MemberNotFoundError` e risponde con `HTTP 404 Not Found`.
4. BlueAgent rimuove il peer da `wg_k`
5. BlueAgent elimina il record del membro dal database SQLite.
6. BlueAgent rilascia il lock per il team k.
7. BlueAgent risponde con stato `HTTP 204 No Content`.

2.5 Requisiti di Sistema

I requisiti di sistema definiscono i vincoli architetturali, infrastrutturali e operativi che BlueAgent deve garantire per supportare i casi d'uso descritti. Di seguito sono formalizzati in maniera sintetica, raggruppati per categoria funzionale con codice univoco `SR-<CATEGORIA>-<ID>`.

2.5.1 Piattaforma Host e Infrastruttura (SR-HST)

- `SR-HST-01` **(Sistema Operativo Host)**: L'host deve eseguire Debian 13 con kernel Linux, IP forwarding abilitato e supporto completo al framework di filtraggio `nftables`.
- `SR-HST-02` **(Storage Pool CoW)**: L'host deve disporre di uno storage pool ZFS con supporto a snapshot e clonazione a livello di blocchi.
- `SR-HST-03` **(Demoni SDN OVN)**: L'host deve eseguire i servizi Open vSwitch e OVN (`ovs-vswitchd`, `ovn-northd`, `ovn-controller`) per il provisioning di switch logici.

2.5.2 Motore di Containerizzazione (SR-ENG)

- `SR-ENG-01` **(Astrazione Runtime)**: Disaccoppiamento della logica di orchestrazione tramite interfaccia astratta `ContainerEngine` per garantire portabilita del runtime.
- `SR-ENG-02` **(Client Asincrono UDS)**: Comunicazione non bloccante con Incus via Unix Domain Socket.

- SR-ENG-03 **(Isolamento Cgroups v2)**: Applicazione perentoria di quote hardware per container (`limits.cpu` e `limits.memory`) per impedire starvation delle risorse.
- SR-ENG-04 **(Clonazione Snapshot CoW)**: Duplicazione istantanea delle istanze container basata su snapshot CoW a livello di storage, azzerando la duplicazione fisica iniziale dei dati.

2.5.3 Rete Ibrida e Connettivita (SR-NET)

- SR-NET-02 **(Linux Bridge L2)**: Uso di bridge L2 a livello kernel per la rete di produzione per garantire prestazioni I/O vicine alla velocita della RAM.
- SR-NET-04 **(Topologia Deterministica)**: Configurazione statica deterministica della rete nei container in base al suffisso per l'indirizzo ip fornito nella configurazione.
- SR-NET-05 **(TCP MSS Clamping)**: Applicazione di regole `nftables` per forzare il clamping del MSS, prevenendo blackhole dovuti alla riduzione di MTU (WireGuard/Geneve).
- SR-NET-06 **(Isolamento e Masquerading NAT)**: Isolamento del traffico tra team differenti via set `@team_links` e mascheramento postrouting del traffico uscente.

2.5.4 Motore di Verifica OVN Sandbox (SR-VRF)

- SR-VRF-01 **(Dark Switch OVN L2)**: Creazione di uno switch logico effimero OVN in fase di verifica, isolato e senza gateway host, per prevenire sovrapposizioni di rotte con il bridge live.
- SR-VRF-02 **(Fedelta Topologica)**: I cloni nella sandbox preservano l'interfaccia `eth0` e la configurazione IP identica a quella di produzione (`10.k.1.x`).
- SR-VRF-03 **(Readiness Probes e Timeout)**: Esecuzione di controlli di `readiness` con tentativi periodici prima della verifica effettiva e gestione esplicita dei timeout sui comandi con restituzione dell'oggetto `VerifyResultResp`.

2.5.5 Control Plane, Concorrenza e Persistenza (SR-CON)

- SR-CON-01 **(Two-Tier Locking)**: Meccanismo di lock a due livelli (`asyncio.Lock` per coroutine e file-lock cross-processo) per prevenire mutazioni concorrenti sulla stessa risorsa (`HTTP 409 Conflict`).
- SR-CON-02 **(Self-Healing Automatico)**: Demone in background `LifecycleManager` che bonifica switch OVN e container di verifica orfani con un periodo di tolleranza (*grace period*) di 90 secondi.

- `SR-CON-03` **(Healthcheck & Telemetria)**: Endpoint pubblico `GET /health` per la diagnostica del servizio e `GET /teams/{team_k}/peers` per monitorare handshake e metriche di traffico dei peer.

2.5.6 Pipeline Immagini Distrobuilder (`SR-IMG`)

- `SR-IMG-01` **(Definizione IaC Distrobuilder)**: Creazione dichiarativa di immagini per container di sistema a partire da file di definizione YAML,

2.6 Esposizione dell'Architettura

Sia $k \in [0, \ldots, n-1]$ con n il numero di team partecipenti.

La topologia e' suddivisa logicamente in più reti:

- **una rete di produzione per ogni team**, dove la rete production-k identifica gli oggetti di rete usati per collegare i container vulnerabili alla rete per il team k
- **una o più reti di verifica, al più una per team**. la rete verify-k identifica gli oggetti di rete ed i container duplicati utilizzati per la verifica richiesta dal team-k

2.6.1 rete production-k

La rete production-k e' fatta cosi, contiene:

- wg-k: dispositivo L3, fa da terminazione per il tunnel del team k esimo. Assegna l'ip `10.k.0.2/32` al client che si collega.
- br-team-k: switch L2. Ha un gateway assegnato in modo da poter permettere all'Host di instradare da reti esterne.
- team-k-profile: profilo Incus per collegare l'interfaccia eth0 del container del team k a br-team-k
- eth0: le interfacce di rete eth0 dei container vengono collegate a br-team-k.
- regole per il forwarding e l'instradamento (`nftables`).

2.6.2 rete verify-k

La rete verify-k e' fatta cosi, contiene:

- **rete OVN** temporanea verify-ovn-k. Implementa uno switch ed un router per instradare verso l'esterno utilizzando le regole NAT

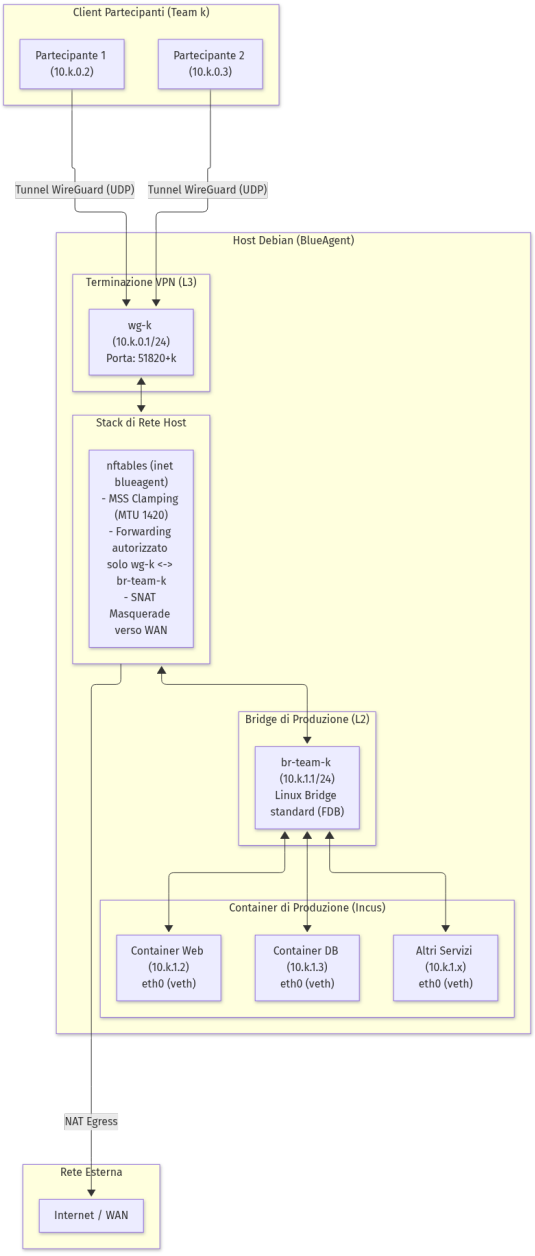


Figura 2.1: Rete Production K

  - ad **incus**: questa rete appare come un'unico oggetto di rete
  - in **OVN**: abbiamo l'implementazione della rete verify-ovn-k. Questa implementazione e' fatta di un router e di uno switch di rete open vswitch gestiti da OVN.

- incusbr0: e' il bridge di rete di incus, qui passa il traffico che dalla rete ovn deve uscire verso l'esterno.
- eth0: le interfacce dei container instradano verso verify-ovn-k. OVN si occupa di capire come instradare i pacchetti.
- **container verificatore**: `verify-runner-xxxx` (IP `10.k.1.250`), istanza effimera che esegue gli script di verifica e gli exploit contro i target duplicati.

2.6.3 vista integrata (merge produzione e verifica)

La coesistenza tra la rete di produzione e la sandbox di verifica su uno stesso host fisico mette in risalto il principio di **fedeltà topologica** e il **totale isolamento del traffico**:

1. **Topologia e Indirizzamento Invarianti**: I container clonati mantengono esattamente lo stesso indirizzo IP statico (`10.k.1.x`) e lo stesso gateway (`10.k.1.1/24`) della produzione.
2. **Zero Conflitti di Routing sull'Host**: Sull'host esiste solo il gateway L3 di `br-team-k`. La rete `verify-xxxx` opera interamente all'interno del datapath di Open vSwitch/OVN senza creare interfacce L3 o rotte in conflitto nella tabella di routing del kernel Linux.
3. **Isolamento Completo (Dark Verification)**: Il traffico di exploit inviato dal `verify-runner` è confinato allo switch virtuale OVN: non può essere intercettato su `br-team-k` o tramite `tcpdump` dai partecipanti, e un eventuale crash distruttivo non ha ripercussioni sui container live.

2.7 Difesa dell'architettura

La progettazione di BlueAgent risponde alle esigenze operative tipiche di scenari reali di Blue Teaming, bilanciando fedeltà dell'ambiente, isolamento dei test ed efficienza delle risorse.

l'architettura di blueagent ha le seguenti proprieta:

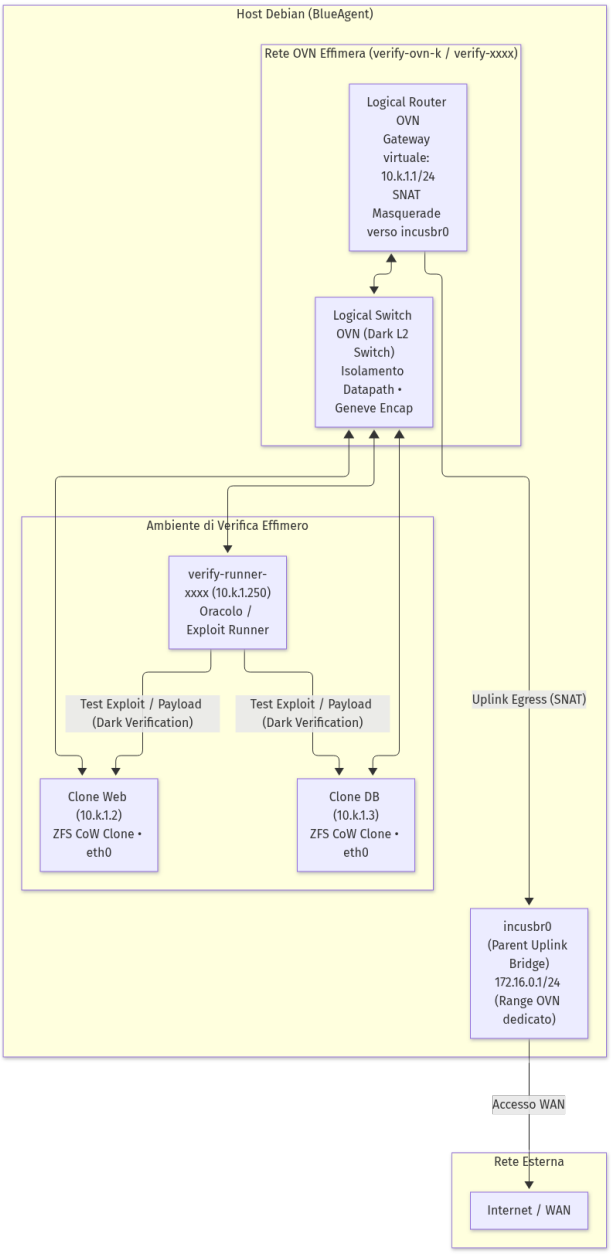


Figura 2.2: Rete Verify K

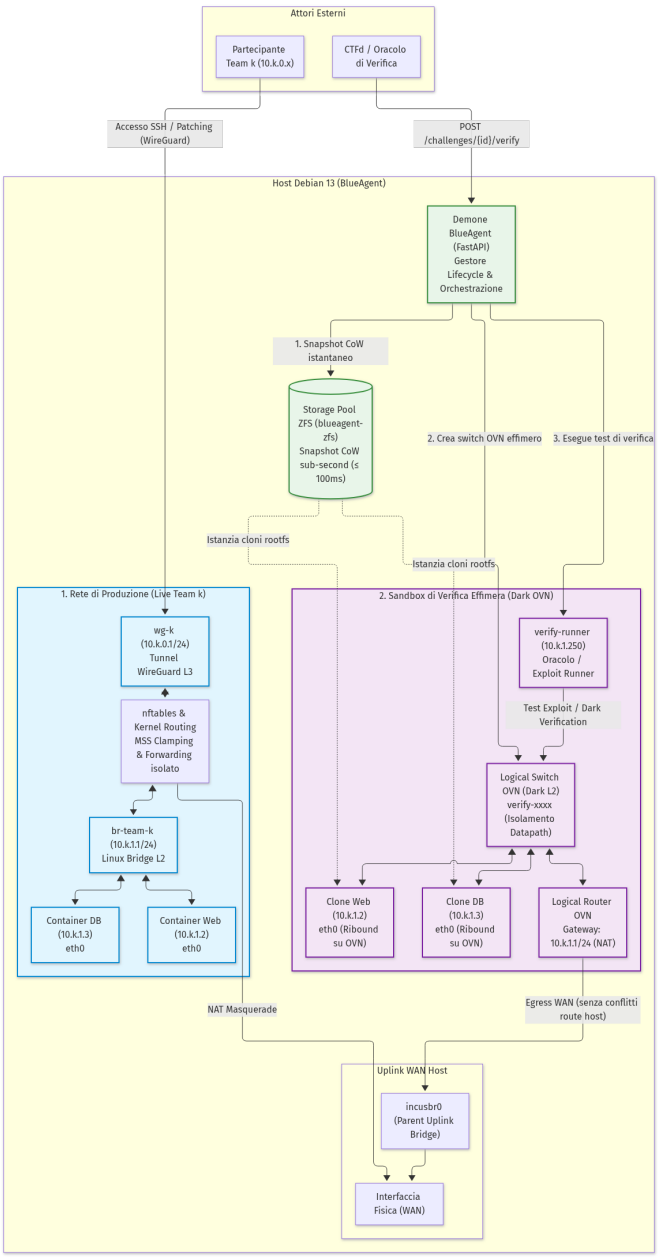


Figura 2.3: Vista Integrata Merge Produzione E Verifica

- **Zero Side-Channel Leakage**: invece di eseguire la verifica sull'ambiente di produzione, Blueagent isola la verifica su cloni effimeri ed una rete ovn inaccessibile. Dunque un partecipante non puo' intercettare il payload di test (accedendo alle interfacce di rete, guardando i log di systema, intercettando le chiamate di sistema)
- **Nessun impatto sull'ambiente di produzione**: Test distruttivi o exploit instabili possono penalizzare l'SLA del team, o di compromettere il lavoro del team salvato nella macchina.
- **Fedelta Topologica**: i container mantengo le interfacce come definite in origine, gli stessi IP statici e lo stesso gateway, sia in produzione che in verifica
- **Ambiente Stateful Completo:** A differenza di soluzioni come kCTF (nsjail/OCI), i container di sistema Incus integrano ambienti con `systemd`, log persistenti e architetture multi-tier (Web + DB), simulando sistemi reali.

inoltre, tecnicamente la clonazione dell'ambiente di produzione e' fattibile poiché:

- clonare un'ambiente con 2 container ubuntu, in un filesystem **ZFS** che offre un meccanismo di **COW**, richiede secondo le misurazioni fatte,all'incirca **700 ms**.
- un singolo container di sistema (es: alpine) richiede all'incirca 100 RAM. (da misurare). dunque e' teoricamente possibile instanziare centinaia di container all'interno di un singolo host che esegue Blueagent.
- la gestione delle risorse morte avviene tramite un demone `LifecycleManager` che distrugge le risorse orfane dopo 90 secondi di inattivita.
- l'uso di reti OVN permette di replicare la topologia di rete in produzione senza generare conflitti nelle tabelle di routing nell'host.

Voglio precisare qui che ho deciso di utilizzare OVN per gestire la rete di verifica per evitare l'overhead dovuto ai tunnel Geneve, a tutto il resto dell'infrastruttura OVN all'interno dell'ambiente di produzione.

2.7.1 4. Destinatari e Casi d'Uso

- **Corsi universitari di sicurezza informatica:** Laboratori pratici di patch management e hardening di sistemi Linux completi.
- **Training aziendale Blue Team / SOC:** Esercitazioni di remediation, analisi forense e configurazione di contromisure su servizi multi-container.
- **Competizioni Attack-Defense e Defense-Only:** Gare con decine di team che richiedono ambienti persistenti a basso costo infrastrutturale.

2.7.2 5. Critica e Limiti Architetturali

Nonostante i vantaggi, l'architettura presenta compromessi e vincoli strutturali:

- **UDP Path MTU Blackhole:** La combinazione di tunnel WireGuard (MTU 1420) e Geneve OVN (MTU 1442, all'interno della **rete di verifica**) impone riduzioni di MTU. Mentre il TCP beneficia del MSS clamping via `nftables`, i pacchetti UDP di grandi dimensioni con flag *Don't Fragment*, potrebbero essere scartati dal kernel.
- **Limite scalabilità a 254 team:** L'indirizzamento deterministico `10.k.x.x` vincola k a un singolo byte ($0 \leq k \leq 254$), quindi ad un massimo di 255 team
- **Concorrenza e blocco multi-nodo:** Il meccanismo `TwoTierLock` (`asyncio.Lock` combinato con `flock`) è confinato al singolo nodo. Per scalare l'architettura su piu nodi bisogna introdurre l'uso di un gestore di lock distribuito come Redis.

2.8 Benchmarks

Capitolo 3

Architetture Alternative

3.0.1 PID namespace separato per la verifica

Si vuole eseguire gli script di verifica direttamente all'interno dell'ambiente di produzione del team, isolando i processi utilizzando un PID namespace separato.

L'obiettivo e' quello di nascondere a tool come `pspy`, `ps` e `top` gli script eseguiti in fase di verifica, evitando che il concorrente, dal suo ambiente possa vedere l'exploit ed i suoi parametri di verifica.

Questa soluzione evita il costo di clonare il container, tutta via e' inadatta al nostro scopo

Il filesystem, resta completamente condiviso:

- se un exploit agisce su un servizio web, questo potrebbe lasciare tracce dell'exploit nei log
- se lo script di verifica o l'exploit crea cartelle temporanee, possono essere lette dal partecipante
- se lo script carica il payload su una porta in cui il servizio vulnerabile e' esposto, allora con `tcpdump` posso ascoltare il traffico nel container.

Quando isoliamo il namespace bisogna anche:

- isolare il mount namespace, per isolare la cartella `/proc`

Tuttavia, tool come `pspy` tramite lo **scanning degli eventi** `inotify` su `/proc`, e lo **scan delle chiamate di sistema**, puo' rilevare l'avvio di nuovi processi.

Inoltre, l'exploit potrebbe essere di **tipo di struttivo**, portando al **crash** di un servizio importante o dell'intero sistema. Si vuole dunque evitare di **interrompere il workflow** del team in fase di verifica.

Se e' abilitata la misurazione dell'**SLA**, per la challenge, far crashare un container vuol dire abbassare l'SLA e dunque penalizzare il punteggio del team.

Per quanto riguarda la rete, un PID namespace separato non implica la creazione di una nuova interfaccia di rete:

- Se lo script verificatore si trova sulla macchina vulnerabile, allora puo' mandare il payload utilizzando l'interfaccia di loopback, che e' accessibile anche ai partecipanti.
- Il partecipante può eseguire `tcpdump -i lo` o ispezionare le connessioni attive con `ss` e `netstat`, catturando il traffico di verifica in chiaro.

Dunque, l'approccio basato su pid namespace e' stato scartato, poiché non isola le altri componenti del sistema.

3.0.2 network namespace separato per la verifica

In blueagent, abbiamo introdotto OVN per semplificare la gestione di due reti con stesso gateway.

Per evitare di aggiungere la complessita computazionale di OVN, si puo' pensare di creare network namespace separato al momento della verifica. All'interno di questo nuovo namespace, chiamato verify-k si crea la rete di verifica composta dal solo bridge L2 e si assegna a questo bridge il gateway originale.

I container duplicati dovranno poi essere collegati al nuovo bridge L2.

Per consentire l'accesso alla rete bisogna usare una coppia di interfacce `veth`, per collegare il namespace nuovo al root namespace. L'inoltro dei pacchetti in questo contesto avviene applicando regole di forwarding e usando il masquerading.

In questo modo i due bridge con stesso ip gateway possono coesistere poiche riesiedono in namespace differenti. In questo contesto ambiguo ci pensa il kernel ad effettuare il routing dei pacchetti verso le giuste interfacce.

Tutta via l'implementazione non e' immediata ed l'uso di Incus non si sposa bene con la creazione di nuovi network namespace dato che Incus non fornisce un'api apposita. Dunque bisogna gestire manualmente le interfacce di rete raggirando Incus.

Quando eseguo il container usando l'interfaccia di Incus, non posso collegare le interfacce di rete dei container al bridge di verifica dentro il namespace verify-k. Questo principalmente perche' il motore di Incus esegue su un namespace separato, dunque e' impossibile che lo veda.

Bisogna dunque raggirare incus, ed istanziare coppie di *veth* che gestiscano questo disallineamento. Ma raggirare incus vuol dire che incus non ha idea dell'esistenza di queste interfacce, creando un disallinamento tra container ed interfacce di rete, che nello sviluppo puo' diventare problematico da gestire.

Dal lato del codice, aumenta la complessita: invece di fare richieste verso l'API di incus per gestire le interfacce, devo eseguire vari comandi per gestire il routing e le coppie *veth*.

3.0.2.1 Conclusione

L'uso di network namespace dedicati sull'host sostituisce l'adozione di un software SDN strutturato con una complessa selva di script manuali di routing e manipolazione delle veth.

L'approccio è stato quindi scartato: l'integrazione nativa di OVN con Incus offre un isolamento di rete standardizzato, manutenibile via API e già predisposto per ambienti distribuiti su cluster.

3.0.3 Esecuzione in-memory con mascheramento

Un'altra alternativa per evitare il costo computazionale e di storage della clonazione dei container consiste nell'eseguire gli script di verifica direttamente all'interno dell'ambiente di produzione, mascherando l'esecuzione su piu fronti, estendendo l'idea vista in 3.3 per creare un'ambiente completamente separato per il payload.

esecuzione in memoria: Si mette lo script o il binario in RAM, utilizzando `memfd_create`, e lo si esegue usando `fexecve`, si evitano file persistenti sul filesystem.

mascheramento di `/proc`: per nascondere il processo in esecuzione a `pspy` si inietta nel sistema una libreria a livello utente `/etc/ld.so.preload` oppure modificando la variabile d'ambiente `LD_PRELOAD`, intercettando le funzioni della lib `C` che leggono dentro `/proc`.

mascheramento dei metadati del processo: sovrascrivere il nome del processo nelle strutture dati del kernel ancora prima che venga avviato e catturato da `pspy`.

tutta via, queste sono tecniche di oscuramento dell'esecuzione e possono essere facilmente mascherate e potrebbero risultare incompatibili con i requisiti dell'utente:

- l'exploit potrebbe totalmente ignorare `LD_PRELOAD`.
- il file `/etc/ld.so.preload` potrebbe essere accessibile allo sfidante, e dunque rimovibile.

a livello kernel, esiste comunque un traccia dell'esecuzione del payload: se l'utente e' in grado di registrare l'esecuzione delle chiamate di sistema allora puo' catturare l'esecuzione del payload.

Anche se l'eseguibile del verificatore risiede in RAM, se questo inoltra richiesta HTTP con all'interno il payload, allora il contenuto puo' essere catturato e letto dallo sfidante.

- il traffico generato puo' essere letto da `tcpdump`
- oppure la richiesta HTTP puo' modificare file accessibili allo sfidante.

l'esecuzione dell'exploit in ambiente separato potrebbe comunque portare in crash il servizio.

dunque, l'esecuzione in-memory con mascheramento di `/proc` si basa su tecniche di evasione fragili ("security through obscurity"). Non offre un vero isolamento: non protegge i file di log, non isola il traffico di rete e non previene il crash distruttivo dei servizi live.

La clonazione tramite ZFS CoW in una Dark Sandbox OVN rimane la soluzione superiore, poiché garantisce un isolamento deterministico a livello di kernel e hypervisor senza ricorrere a trucchi di evasione in user-space.

Capitolo 4

Appendice

4.1 linux network: `net_device`

4.2 linux network: `net_device` e `sk_buff`

fonte: Linux Kernel Networking - Implementation and Theory.pdf

4.2.1 network device drivers

Network device drivers: operano sempre a livello 2 nello stack di rete, ed e' rappresentato nel kernel dalla struttura `net_device`. Questa struttura dati contiene varie informazioni:

- `IRQ`: numero identificativo del dispositivo
- `MTU`: la MTU del dispositivo
- `MAC`: MAC address del dispositivo
- **nome**: `eth0`, `enp0s3`, `wlan0`, ecc...
- `net_device_ops`: contiene i puntatori alle funzioni per la gestione del dispositivo (start, stop, transmit, receive)

Cosa fa un device driver?

- **riceve** i pacchetti destinati al local host, e li inoltra al livello di rete, e poi al livello di trasporto.
- **trasmette** i pacchetti generati dal local host, o fa il forward dei pacchetti ricevuti

routing: ogni pacchetto in in transito in tutti e due i sensi triggera un lookup nel sottosistema di routing, che decide cosa bisogna fare con quel pacchetto. La fase di routing si distingue in 5 fasi, detti **hook**, dove possiamo registrare dei **callback** per il **sottosistema netfilter**.

Per esempio l'hook `NF_INET_PRE_ROUTING`, registra la chiamata al callback per essere chiamato prima che il lookup del sottosistema d routing venga effettuato.

verdetto: ogni callback ritorna un verdetto sul pacchetto decidendo se bisogna accettarlo (`NF_ACCEPT`), oppure buttarlo (`NF_DROP`).

> con `nftables` applichiamo le regole per il routing manipolando quindi strutture dati interne al kernel.

oltre ai callback, **molti altri fattori** influenzano il modo in cui il routing viene effettuato. (es: *il TTL viene decrementato e arriva a 0*).

4.2.2 Socket Buffer

```
struct sk_buff {
    . . .
    struct sock *sk;
    struct net_device *dev;
    . . .
    __u8 pkt_type:3,
    . . .
    __be16 protocol;
    . . .
    sk_buff_data_t tail;
    sk_buff_data_t end;
    unsigned char   *head,
                    *data;
    sk_buff_data_t transport_header;
    sk_buff_data_t network_header;
    sk_buff_data_t mac_header;
    . . .
};
```

- `head` e `data`: sono puntatori, rispettivamente ad `header` e `data` per il layer di rete corrente.

Socket buffer e' la struttura dati su cui tutti i layer di rete nel kernel operano.

4.2.3 Netfilter

fonte: `https://www.digitalocean.com/community/tutorials/a-deep-dive-into-iptables-ar`
fonte:`https://linuxhandbook.com/iptables-vs-nftables/` E' un sottosistema del kernel che fornisce le seguenti funzionalita:

- selezione e filtraggio dei pacchetti

- NAT
- tracciamento delle connessioni
- accounting di rete.

Per quanto riguarda il filtraggio dei pacchetti, abbiamo a disposizione 5 hook differenti:

- `NF_INET_PRE_ROUTING`: e' chiamato nel metodo `ip_rcv()`, ancora prima di entrare nel sottosistema di routing.
- `NF_INET_LOCAL_IN`: e' chiamato nel momento in cui il pacchetto e' indirizzato al local host.
- `NF_INET_FORWARD`: e' chiamato dopo che e' stato effettuato il lookup nel sottosistema di routing
- `NF_INET_POST_ROUTING`: e' chiamato per i pacchetti che devono essere inoltrati, pochi istanti prima che il pacchetto entri fisicamente nel cavo.
- `NF_INET_LOCAL_OUT`: e' chiamato per i pacchetti creati nel local host, prima che escano.

Molte utility, comunemente usate, si basa sul sottosistema netfilter:

- `iptables`: agisce come front-end per netfilter ed implementa un **firewall**. Fornisce un'interfaccia per aggiungere, eliminare, regole netfilter, aggiungere tabelle, ecc...
- `nftables`: e' sviluppato da **netfilter** (l'organizzazione dietro al sottosistema netfilter). E' un rimpiazzo per iptables che soffre di problemi di scalabilita e performance.

iptables: utilizza le **tables** per organizzare le regole da applicare, suddividendo le regole in classi di **table**.

- `nat` table: usata per implementare le regole NAT, ossia si modificano sorgente e destinazione dei pacchetti in transito nello stack di rete per modificarne il comportamento durante il routing.
- `mangle` table: altera gli header IP. (es: modificare il TTL, marcare il pacchetto)
- `raw` table.
- `security` table: per inserire contesto SELinux all'interno dei pacchetti.

nftables: svolege la stessa funzione di iptables, ma usando una sintassi differente. rispetto ad iptables, nftables e' piu efficiente in memoria ed in CPU ed e' piu flessibile.

4.2.4 Connection Tracking

Spesso le connessioni a livello applicativo hanno una connessione mantenuta nel tempo ed una serie di eventi che determinano la storia della comunicazione. Connection Tracking permette al kernel di tenere traccia delle sessioni.

Una Connection Tracking entry nel kernel appare cosi:

```
1  struct nf_conn {
2      /* Usage count in here is 1 for hash table/destruct timer, 1
           per skb,
3      plus 1 for any connection(s) we are 'master' for */
4      struct nf_conntrack ct_general;
5      spinlock_t lock;
6      /* XXX should I move this to the tail ? - Y.K */
7      /* These are my tuples; original and reply */
8      struct nf_conntrack_tuple_hash tuplehash[IP_CT_DIR_MAX];
9      /* Have we seen traffic both ways yet? (bitset) */
10     unsigned long status;
11     /* If we were expected by an expectation, this will be it */
12     struct nf_conn *master;
13     /* Timer function; drops refcnt when it goes off. */
14     struct timer_list timeout;
15     . . .
16     /* Extensions */
17     struct nf_ct_ext *ext;
18 #ifdef CONFIG_NET_NS
19     struct net *ct_net;
20 #endif
21     /* Storage reserved for other modules, must be the last member
           */
22     union nf_conntrack_proto proto;
23 };
```

e' la struttura dati fondamentale per implementare il NAT. Dunque per ogni connessione che deve usufruire del NAT deve tenere in memoria una connection Tracking entry.

4.3 TODO: NAT e masquarade

4.4 bridge

fonte: libro!

Un bridge e' un dispositivo che capisce il protocollo del livello di collegamento, effettua l'inoltro dei dati copiandoli frame per frame e deve essere in grado di mantenere nel suo buffer almeno un frame per volta.

Un bridge assegna un'indirizzo a livello di collegamento ad ognuna delle sue interfacce ed effettua il forward verso di queste se il bridge non e' il destinatario del pacchetto.

- il bridge, se implementa delle funzionalita al livello L3 potrebbe passare il frame al livello superiore e processarlo.
- oppure potrebbe passare il frame ad un protocol handler

utilizzo: un bridge viene di solito usato per unire due LAN differenti.

definizione di host: un qualsiasi device che opera e capisce ad un livello più alto del layer di collegamento.

address learning: un bridge fa *passive learning*. Ipotizziamo che `A` invii un frame verso `B` e che `B` lo riceva poiche `A` e `B` si trovano nella stessa LAN1. Il bridge, riceve anche lui una copia del frame, sulla porta LAN1 e lo inoltra verso LAN2 (ance se non serviva)

- `learning`: ora il bridge sa che sulla porta LAN1 si trova `A`
- ora se `B` invia un frame verso `A`, il bridge sa che `B` si trova nella stessa rete di `A` e non deve copiare il frame su LAN2.

broadcast: se un bridge riceve un frame con destinazione FF:FF:FF:FF:FF:FF, il frame viene copiato su tutte le porte.

aging: quando imparo l'indirizzo di un nuovo host, aggiungo un timer di aging. Quando il timer scade, rimuovo l'indirizzo dalla tabella, se arriva un nuovo pacchetto prima della scadenza, estendo il timer.

4.5 linux bridge

E' un device virtuale: finché non viene collegato ad altri device non puo' ne ricevere e ne trasmettere frame.

Supponiamo di voler implementare questa topologia, nel kernel:

Ossia, data una macchina con le tre interfacce fisiche `eth0, eth1, eth2`vogliamo aggregare le prime due in un'unica interfaccia in modo che questa possa instradare verso `eth2`.

Il kernel dovra:

- al livello del router, vedere le due sottoreti
- effettuare il bridge tra eth0 ed eth1 anche se ho piu interfacce in pratica

enslaving: quando si crea un bridge, bisogna assegnare a questo delle interfacce dette **enslaved**.

Dunque possiamo creare il bridge `br0` a cui assegnamo le interfacce `eth0` ed `eth1`. Dato che `eth0` ed `eth1` sono interfacce al bridge, non hanno bisogno di essere viste a livello 3, e dunque non hanno un'indirizzo IP.

Si assegna un'indirizzo IP all bridge per renderlo partecipe nella sotto rete, in modo che possa ricevere i pachetti da cui estrarre i frame da inoltrare.

In pratica, quando tento di trasmettere su `br0`, il device driver di `br0` gestisce i device **enslaved** effettuando il forwarding dei frame.

Posso eventualmente nell'host, assegnare un'indirizzo IP anche alle interfacce enslaved. Da quel momento il kernel linux si occuperà di instradare verso l'interfaccia si attraverso `br0`, che attraverso il layer 3.

> [!error] eventualmente approfondire Capitolo 8, 10 e 11 di Understanding Linux Network Internals.pdf

4.5.1 STP

Lo Spanning Tree Protocol serve a prevenire loop all'interno della rete nel kernel. Con STP abilitato, i bridge si scambiano Bridge Protocol Data Units in modo da identificare un bridge root e l'interfaccia da bloccare per rendere la topologia della rete libera da cicli.

cicli: all'interno di una rete, a livello L2 puo' causare ondate di messaggi.

4.6 cosa approfondire

4.6.1 shadow bridge

- differenze tra switch L2 e router L3
- risoulzione next-hop e forwarding information base: Quando un container vuole comunicare con un IP esterno alla propria subnet (es. DNS `1.1.1.1` o repository di pacchetti), la Forwarding Information Base (FIB) del container seleziona il default gateway (`10.k.1.1`) e invia una richiesta ARP broadcast (`who-has 10.k.1.1`).
- FIB collision: accade quando sistenta di assegnare `10.k.1.1` a due bridge

4.6.2 dual nic

- multi-homing: due interfacce che instradando verso gateway distinti
- 3. **Policy Routing & Source-Based Routing (`ip rule`)**:

  - Per far funzionare due interfacce con due default route, bisognerebbe iniettare regole di routing avanzato nel container basate sull'IP sorgente (`ip rule add from ... table ...`). Questo viola la semplicità di configurazione e sporca l'ambiente del concorrente.
  - fedalta topologica: aggiungere una seconda interfaccia sporca il comportamento di demoni che usano l'interfaccia `0.0.0.0`
  - `veth`: numero raddoppiato.

4.6.3 PID namespace separato

1. PID vs mount namespace: isolare il PID namespace non isola il filesystem `/proc`
2. `pspy` usa `inotify`: qualsiasi directory creata in `/proc` viene cattuarata da `pspy`
3. file system unico: il partecipante puo analizzare i log ed estrarre ventualmente il payload
4. Network Sniffing su loopback: se i processi condividono lo stesso network namespace, allora il partecipante puo' analizzare il payload se transita su un'interfaccia di rete.
5. il payload potrebbe far carashare il server in esecuzione nell'ambiente dello sfidante

4.6.4 network namespace separato

1. `CLONE_NEWNET`, `ip netns` crea un'istanza isolata dello stack di rete. permette a due brdige di avere lo stesso IP
2. `veth`: per dare connessione serve una coppia di interfacce `veth` con `ip forwarding` e `SNAT/MASQUERADE`
3. creazione di interfacce non tracciate da incus, disallinamento
4. `rtnl_lock`: creare e distruggere network namespace comporta il lock sul routing globale

4.6.5 esecuzione in memoryu

- **dynamic linker hooking**: sovrascrivere la libreria `C` per filtrare e nascondere `/proc`
  - funziona solo con binari linkati dinamicamente
  - `/etc/ld.so.prelaod` puo' essere modificato
- **fileless execution**: `memfd_create`, crea un descrittore di file anonimo residente unicamente in RAM (memoria volatile protetta).
  - `fexecve` lo esegue

- usando filtri `eBPF` posso a livello kernel intercettare le chiamate di sistema
- principio di saltxzer e schroeder:
  - l'isolamento deve basarsi su barriere architetturali matematicamente verificate (hypervisor, namespaces, CoW snapshots), non su trucchi di evasione ("security through obscurity").
    * L'occultamento non previene il salvataggio dei log applicativi, non isola il traffico di rete e non evita il crash distruttivo del servizio.

4.7 VPN wireguard

fonte: `https://www.wireguard.com/papers/wireguard.pdf` **vpn**: `https://ieeexplore.ieee.org/document/4756450`

VPN: e' un modo per comunicare in modo sicuro tra membri di un gruppo attraverso l'uso di una infrastruttura di comunicazione pubblica, grazie all'uso di un protocollo di tunneling.

Wireguard e' un VPN che utilizza il concetto di **cryptokey routing**: viene mantenuta in memoria una tabella dove ad ogni chiave pubblica di un peer, viene associata una lista di indirizzi ip identificati come allowed. Fornisce un'interfaccia **stateless** per utilizzare il **tunnel VPN**, ossia un'interfaccia di rete virtuale, che potrebbe essere per esempio `wg0`.

> per stateless, si intende il fatto che l'amministratore non si deve occupare di gestire lo scambio delle chiavi, connessioni, disconnessioni, poi che viene tutto gestito dietro le quinte.

Rispetto ad OpenVPN, Wireguard e' percepito come più sicuro ed efficiente:

- OpenVPN utilizza TLS ed espone molte delle sue funzionalità, aumentando la superficie di attacco sulla VPN.
- OpenVPN utilizza interfacce TUN/TAP: agiscono in user space ma al costo di penalizzare di molto le performance, infatti i pacchetti devono essere copiati piu volte nel passagio da user space a kernel space.

Scambio di chiavi: Wireguard non utilizza un meccanismo specifico per scambiare le chiavi. Infatti Wireguard non include un protocollo interno per la distribuzione delle chiavi, si delega lo scambio a meccanismi esterni e preesistenti (come SSH)

Cosa vuol dire in pratica? Lo scambio delle chiavi si fa per esempio manualmente, si copia e incollano manualmente le chiavi pubbliche nel file di configurazione di wireguard oppure si usano tool automatizzati.

vediamo un esempio di *cryptokey routing table*: questa decide in pratica quali coppie (Peer Public Key, Allowed Source IP) possono instradare nell'interfaccia wireguard.

opzionalmente e' possibile specificare verso quale endpoint inoltrare il pacchetto, ma almeno un peer, ad esempio il client, deve conoscere l'endpoint a cui inviare i datagrammi UDP crittografati.

> l'**endpoint** identifica l'indirizzo ip e la porta UDP su cui il demone WireGuard e' in ascolto.

Brevemente il flusso dei dati e' il seguente: l'interfaccia `wg0` riceve un pacchetto

- il client incapsula il pacchetto all'interno di una datagramma UPD il cui payload viene crittografato utilizzando `ChaCha20Poly1305`
- il client invia il datagramma all'endpoint specificato, oppure imparato dal Wireguard in precedenza.
- il server riceve il dataramma, verifica la correttezza nella *cryptokey routing table*
- il server autentica e decripta il messaggio ed estrae il pacchetto originario
- il pacchetto originario viene inserito in una coda nell'interfaccia wg0

4.8 SDN

`https://networklessons.com/network-automation/introduction-to-sdn-software-defined-`
`https://ridhanegara.staff.telkomuniversity.ac.id/files/2017/04/Thomas-D.`
`-Nadeau-Ken-Gray-SDN-Software-Defined-Networks-O_039_Reilly-Media-2013.`
`pdf`

una SDN opera su tre differenti piani:

- piano di controllo
- paino dei dati
- piano di management

Il **piano di controllo** e' responsabile per lo scambio di informazioni sul routing, le tabelle ARP, il routing BGP, OSPF, ed eseguire l'arlgoritmo STP per creare una topologia senza cicli.

Il **piano dei dati** e' responsabile di instradare il traffico, ossia di effettuare il forward dei pacchi, e lo fa applicando le informazioni in possesso dal piano di controllo. Dunque, tra le verie operazioni puo: incapsulare e decapsulre i pacchetti, fare il match guardando MAC o indirizzi o IP, applicare le regole NAT e oppure fare il drop dei pacchetti.

Il piano di management viene usato per accedere e gestire i device di rete: si accede al device usando telnet, SSH, o altri servizi.

Nella SDN utilizziamo un controller centrale per applicare il piano di controllo. Il controller potrebbe essere su un host fisico oppure su una macchina virtuale.

Cio' implica che il contorller decide come saranno fatte le tabelle di inoltre, e gli switch ed i router si limitano ad eseguirle. Si toglie di mezzo la complessità dovuta al setup di migliaia di interfacce, dato che ora la gestione e' centralizzata. Tuttavia il controller SDN e' incaricato di fornire i dati necessari all'inoltro a tutti i device.

Per implementare questo meccanismo ci sono due interfacce:

- northbound
- southbound

southbound: Il controller SDN deve poter comunicare con i device di rete in moda da programmare il piano dei dati. La southbound interface permette la comunicazione. E' un'interfaccia software che puo' essere implementata in vari modi.

northbound: e' l'interfaccia usata per accedere al controller SDN. Permette di applicare modifiche e di accedere ad informazioni sul sistema

- Ottenere una lista di tutti i device di rete preenti
- Aggiungere una nuova VLAN agli switch
- Mostrare la topologia della rete
- Configurare gli indirizzi ip

  abbiamo accenato prima ad Open vSwitch. Riprendendo il discorso, il controler SDN non fa altro che comandare gli switch gestiti da Open vSwitch.

4.9 ZFS

fonte: `https://openzfs.org/wiki/System_Administration` `https://blog.victormendonca.com/2020/11/03/zfs-for-dummies/`

Uno dei principi fondamentali di ZFS:

> Un blocco in uso non viene mai sovrascritto.

Modificare dati vuol dire dunque scrivere il nuovo contenuto in una nuova porzione di memoria, ed il filesystem dovra puntare a questa nuova area.

Questo principio, permette di implementare e gestire facilmente feature piu "avanzate" come snapshots ed il self-healing.

4.9.1 pool

Una pool e' come un grande albero:

- le foglie contengono i dati
- i nodi interiori sono dei **block pointers**

block pointer: un block pointer tiene traccia di in quali record si trova il figlio, la grandezza e la **checksum**.

- **checksum**: un block pointer contiene la checksum dei nodi figli

uberblock: e' il nodo radice, tutta la pool e' navigabile a partire da qui.

modificare i dati nel filesystem vuol dire ricostruire la catena dei padri a partire dalla foglia:

- D_4 viene liberato, a meno che uno snapshot non lo contenga.]
- D'_4 e' il blocco D_4 modificato.
- La modifica a D_4 scatena il ricalcolo della checksum e il cambio dei puntatori al blocco dati in ind 2
- La modifica alla checksum in ind2 scatena il ricalcolo delle checksum di root
- ecc.

dopo la modifica otteniamo uberblock', la nuova radice che descrive il filesystem con i dati D'_4. L'intera operazione descritta e' **atomica**.

Il costo per una modifica nella pool e' proporzionale alla profondita dell'albero, che non e' necessariamente legato alla grandezza in byte della pool. Il vecchio albero inoltre risiede ancora in memoria per poterne effettuare uno snapshot

4.9.2 transaction groups

Le transazioni sul filesystem sono raggruppate in transaction groups: **txg**. L'intero gruppo viene eseguito atomicamente.

Se un tipo di transazione accumula troppi *dirty data* oppure fa scattare un time-out (di default di 5 secondi), la transazione viene annullata ed il file system ritorna all'ultimo albero per cui e' stato fatto il commit.

> L'ultimo commit potrebbe non essere esattamente l'albero con i dati piu recenti. Tutta via analizzando il log prodotto dal file system: ZFS Intent, si possono recuperare le differenze.

4.9.3 vantaggi di pool e transazioni

Gli snapshot del filesystem vengono eseguiti a costo quasi zero. Fare uno snapshot vuol dire, chiedere al filesystem di non liberare i dati delle foglie se questi devono essere eliminati o modificati in versioni future dell'albero.

Clonare costa poco in memoria: basta un riferimento allo stesso blocco.

ZFS tiene traccia per ogni blocco del **txg** che lo ha creato, dunque ZFS puo' facilemente identificare quali blocchi sono stati creati tra uno snapshot e l'altro.

ZFS e' in grado di identificare gli erorri nei blocchi grazie al controllo della corrispettiva checksum che risiede nel padre. Se nella pool esiste una copia, ridondante dei dati, allora ZFS puo' automaticamente ripristinarne il contenuto.

4.9.4 quando ZFS costa in memoria?

Frammentazione: modificare un file vuol dire allocare qua e la nella pool nuovi blocchi. Dunque modificare un blocco in ZFS vuol dire rompere la sequenzialità dei blocchi nel file system. Ne consegue che le performance di ZFS dipendono da quanto spazio libero c'e' nella pool.

> Se la pool e' vuota allora trovare spazio contiguo e' facile, altrimenti l'allocatore deve lavorare di piu per trovare spazio per nuovi dati.

Per fare qualsiasi modifica, anche un'eliminazione, ZFS ha bisogno di memoria.

Per fare una modifica parziale ad un record, bisogna leggerlo, modificarlo e scriverlo dentro un nuovo record. Dunque piccole modifiche possono essere molto costose.

4.10 struttura di una pool

4.10.1 `vdev`

Un vedv (virtual device) e' un blocco fondamentale in ZFS. E' un raggruppamento logico di device di dati fisici (hard disk, ssd, ecc...)

una **foglia vdev** e' il un tipo di vdev che corrisponde ad uno storage fisico, ossia l'endpoint nella gerarchia dei tipi di storage in ZFS.

I `top-level vdevs`, sono i device figli del `root vdev`. Possono essere device singoli, oppure gruppi logici che raggrupano piu **foglie** `vdevs`.

La `root vdev` si trova in cima alla gerarchia della pool, ed aggrega i vdev figli in un'unica unita di storage, ossia la pool.

Ci sono diversi tipi di `vdevs`:

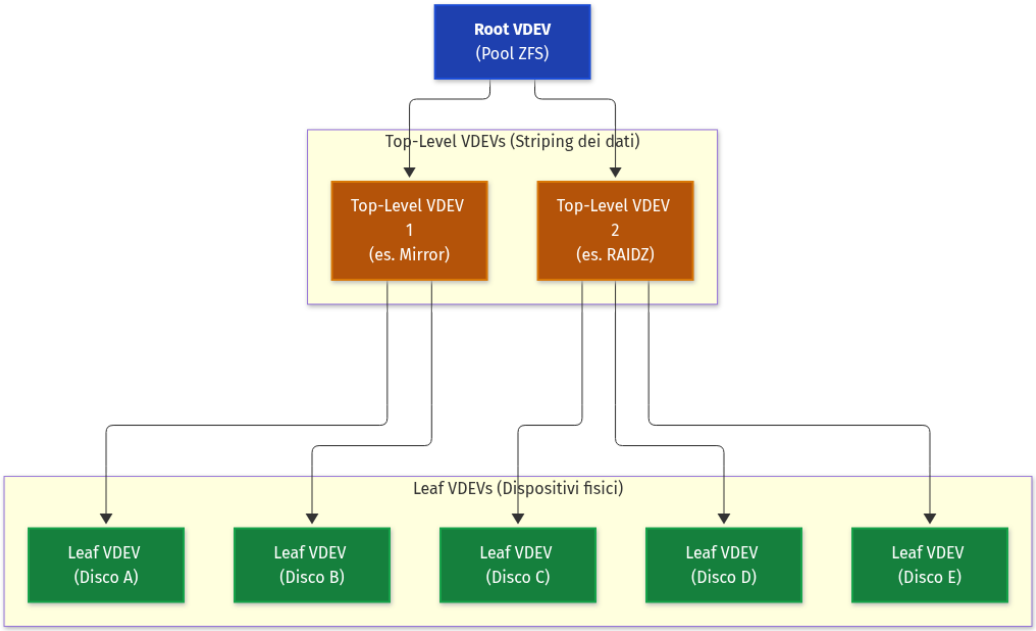


Figura 4.1: Vdev

- **single device**: rappresenta un singolo disco, file o partizione usata a livell di top-level vdev. non dispone di meccanismi di ridondanza.
- **mirror**: e' un vdev che tiene in memoria gli stessi dati su uno o piu dispositivi per ridondanza.
- **RAIDZ**: un vdev che offre ridondanza e fault tollerance.
- **dRAID**: un vdev che offre parita distribuita.

Ci sono altri tipi di `vdev`, tra cui:

- **Spare**: un dispositivo che rimpiazza dispositivi che falliscono.
- **Cache**: un dispositivo utilizzato per fare il cache di dati utilizzati frequentemente nalla pool. Mantiene le copie dei dati gia presenti nella pool, dunque se un giorno esplode non e' un problema.

4.10.2 quindi a che serve un vdev?

I vdev sono i blocchi fondamentali di una pool ZFS. Il modo in cui i vdev sono orchestrati determinano le performance del sistema, la capacita e la ridondanza dei dati.

4.11 dataset

Una pool e' una pila di spazio, i dataset descrivono il modo in cui lo spazio viene suddivso. Creare un dataset e' un'operazione leggera e non comporta allocazione.

tipi di dataset:

- **file system**: un file system POSIX montato-.
- **volume**: un dispositivo a blocchi usato per macchine virtuali swap e file system non supportati da ZFS.
- **snapshot**.

i dataset formano una **gerarchia**. questa gerarchia rispecchia i nomi dei dataset e le proprieta vengono ereditate da questa gerarchia.

```
zfs create pool/home
zfs create pool/home/alice
zfs create -V 40G pool/vm/disk0         # a zvol

zfs list                                # file systems and volumes
zfs list -t all -r pool                 # everything, recursively
zfs destroy pool/home/alice
```

4.12 snapshots

creare uno snapshot vuol dire creare un albero che punta ai blocchi di dati che erano "**in vita**" al momento dello snapshot.

Uno snapshot e' una copia read-only del sistema o di un volume ad un determinato istante. Creare uno snapshot e' un'operazione quasi istantanea, non consuma spazio ed e' atomica anche se ricorsiva (ossia sono coinvolti piu dataset)

uno snapshot e' raggiungibile tramite la cartella `.zfs/snapshot`

> il restore di un file da uno snapshot si fa in questo modo: `cp /home/user /.zfs/snapshot/.../notes.txt /home/user/notes.txt`

il comando `zfs rollback` permette di scartare le modifiche effettuate dall'ultimo snapshot e di tornare a quel determinato stato.

4.12.1 clones

Un clone e' un dataset scrivibile, i cui contenuti iniziali sono quelli di uno snapshot. Come uno snapshot, il clone inizialmente non consuma memoria, e poi comincia a divergere rispetto all'origine con l'aumentare delle modifiche effettuate.

4.12.1.1 nsjail

Nsjail e' una sandbox sicura che esegue un server TCP. Ad ogni connessione viene fatto il fork e dell'ambiente e rilasciato all'utente per l'utilizzo. L'ambiente separato

rende impossibile agli utenti di interferire tra di loro sebbene operino nello stesso sistema.

Nsjail e' un tool che deve essere configurato per ogni challenge, specificando quali file dovrebbero essere esposti nel nuovo ambiente, e quali limiti applicare.

Nsjail riesce a isolare i processi utilizzando namespaces, resource limits ed filtri seccomp.

4.12.1.2 namespaces

4.12.1.3 seccomp bpf

Permette ad un processo di specificare un filtro da applicare alle chiamate di sistema in uscita, specificando un comportamento (es: uccidere il processo) quando chiamate filtrate vengono richieste dal processo.

```
1 scmp_filter_ctx ctx;
2 seccomp_rule_add(ctx, ...);
3 seccomp_load(ctx, ...);
```

4.12.1.4 cgroups

e' un meccanismo per aggregare o partizionare i task in gruppi gerarchici. i task sono racchiusi in sottosistemi, dove un sottostistema si occupa di schedulare le risorse sottostando alle limitazioni.

`cgroups` vengono utilizzati per applicare limitazioni e fare accountig su gruppi di task.

Elenco delle figure

1.1 Kctf Architecture 5
1.2 Ctfbox Architecture 8
1.3 Cytrone Architecture 10
1.4 Cytrone Training 11
1.5 Cytrone Cyberrange 12
1.6 Cytrone Performance 14
1.7 Cytrone Largescaleperformance 15
1.8 Kypo Basic Elements Sandboxes 17
1.9 Kypo Topology Instance 18

2.1 Rete Production K 34
2.2 Rete Verify K 36
2.3 Vista Integrata Merge Produzione E Verifica 37

4.1 Vdev 56

Elenco dei codici

Elenco degli algoritmi